Byzantine Consensus in the Partially Authenticated Setting

CHRISTOPH LENZEN, Aalto University, Finland
JULIAN LOSS, Ruhr University Bochum, Germany
KECHENG SHI, CISPA Helmholtz Center for Information Security, Germany and Saarland University, Germany
BENEDIKT WAGNER, Ethereum Foundation, Switzerland

Byzantine Agreement and Broadcast are traditionally studied in one of two extremes: the authenticated setting, where a public key infrastructure (PKI) enables universally verifiable signatures and yields higher fault tolerance, and the unauthenticated setting, where no PKI is available and resilience necessarily drops. Motivated by Proof-of-Stake blockchains, where only a stable subset of participants (e.g., validators) have registered long-term keys while others do not, we initiate a systematic study of consensus in the *partially authenticated* setting, where a subset of parties are *registered* in a PKI and the remaining parties are *unregistered*.

We provide a nearly complete feasibility characterization of the resilience as a function of the number s of registered parties among n total parties. First, we show that Byzantine Agreement or Byzantine Broadcast with an *unregistered* sender is possible if and only if $t \leq \max\{\lceil s/2 \rceil, \lceil n/3 \rceil\} - 1$, matching a simple protocol and an impossibility bound. Second, for Byzantine Broadcast with a *registered* sender, we give a deterministic synchronous broadcast protocol tolerating up to $t \leq s + \lceil (n-s)/3 \rceil - 1$ Byzantine faults (equivalently, $3t < n + 2s$); while we present the binary case in the main body for clarity, our techniques extend to an efficient multivalued protocol. We complement this with a matching lower bound in a strengthened leakage model in which the adversary learns each party's private state at the end of every round, ruling out both deterministic protocols and randomized protocols that rely only on short-lived secrets and the basic signing/verification interface.

Authors' Contact Information: Christoph Lenzen, Aalto University, Finland; Julian Loss, Ruhr University Bochum, Germany; Kecheng Shi, CISPA Helmholtz Center for Information Security, Germany and Saarland University, Saarbrücken, Germany; Benedikt Wagner, Ethereum Foundation, Switzerland.

1 Introduction

Byzantine Agreement and Broadcast [22] are fundamental problems in distributed computing that ask n parties to agree on a common and non-trivial output by running a joint protocol Π. Crucially, Π should remain secure and live even in the presence of up to $t < n$ *Byzantine* parties that may deviate from the protocol arbitrarily.

A long line of literature has studied protocols for the *synchronous model* in which all messages are delivered within a known upper bound Δ on network delay and protocols proceed in lock-step *rounds*. Existing work mainly considers two types of protocols:

Authenticated protocols. In the authenticated setting, parties share a pre-established public key infrastructure (PKI) that assigns to every party P_i a public key pk_i. Using its corresponding secret key sk_i, P_i can generate a signature that everyone can efficiently verify using pk_i. Under these conditions, it is well-known that deterministic synchronous Byzantine Agreement is efficiently solvable tolerating t corruptions if and only if $2t < n$ and deterministic synchronous Byzantine Broadcast is solvable for any $t < n$ corruptions [10, 20].

Unauthenticated protocols. In the unauthenticated setting, parties do not have access to a PKI and it is assumed only that parties are connected via pairwise authenticated channels. It is well-known that deterministic synchronous Byzantine Agreement and Broadcast are possible if and only if $3t < n$ [5, 20].

Partially Authenticated Protocols. A natural question is what resilience a protocol can achieve when a subset $\mathcal{S} \subseteq \mathcal{P}$ of *registered* parties holds a public key, whereas the remaining parties $\mathcal{N} := \mathcal{P} \setminus \mathcal{S}$ are *unregistered* parties and have no registered key. We refer to this setting as the *partially authenticated setting*. Indeed, this setting was already explored in the pioneering work of Dolev and Strong [10], who showed that broadcast is possible if $|\mathcal{S}| \geq t + 1$. It immediately follows that Byzantine Agreement is possible if $|\mathcal{S}| \geq t + 1$ and $2t < n$, and the latter condition is trivially necessary. Protocols that assume such a *partially authenticated PKI* have recently come into focus due to practical motivation from the blockchain space. While dynamic participation models capture the temporal fluctuations of permissionless networks, they often abstract away the cryptographic asymmetry inherent in Proof-of-Stake blockchains such as, e.g., the Ethereum blockchain. Such systems typically distinguish two types of nodes: 1) *validator nodes* who participate in proposing and voting on blocks and hold *registered* long-term public keys[1] and 2) *non-validator nodes* who may join and leave freely without *registering* a public key. This shows that in reality, participation is not just about being "online"; it is also about which parties have keys and which parties do not. In this work, we give a nearly complete characterization of Byzantine Agreement and Byzantine Broadcast in the partially authenticated setting with $s = |\mathcal{S}|$ by showing the following results.

- Byzantine Agreement or Byzantine Broadcast with an *unregistered sender* $P_S \in \mathcal{N}$ is possible if and only if $t \leq \max\{\lceil s/2 \rceil, \lceil n/3 \rceil\} - 1$. Our upper bound is simple (see Appendix A): if the maximum is attained by $\lceil n/3 \rceil$, we can run a standard protocol ignoring the PKI. Otherwise, registered parties in $\mathcal{S}$ decide their output by running a consensus protocol among themselves. They then send their output to unregistered parties in $\mathcal{N}$ who decide their output by taking a majority vote on the messages they received from registered parties in $\mathcal{S}$. Our lower bound shows that this is the tight protocol for this setting.
- We then turn our attention to the much more interesting and challenging problem of Byzantine Broadcast with a registered sender $P_S \in \mathcal{S}$. On the positive side, we give a broadcast protocol for a registered sender $P_S \in \mathcal{S}$ that is secure against up to $t \leq s + \lceil (n - s)/3 \rceil - 1$ Byzantine corruptions. We complement this result with a matching lower bound

[1] This is usually achieved by validators staking funds.

for a setting where the private state of a party is leaked to the adversary at the end of each round r (including random coins sampled in round r). Since long-term keys do not exist in this model, signatures are instead provided exclusively through a suitable idealized functionality $\mathcal{F}_{\text{sign}}^{\mathcal{S},\mathcal{N}}$. Our bound rules out deterministic protocols as well as randomized R-round ones that (i) succeed with probability $1 - o(1/R)$ and (ii) rely only on short-term secrets and the basic properties of a signature scheme, i.e., the ability to sign a message and verify a signature. Interestingly, this probability threshold is asymptotically tight: we show that a simple protocol achieves success probability $1 - O(1/R)$ based on a shared coin.

1.1 Technical Overview

As already explained above, we focus here exclusively on the interesting case of Byzantine Broadcast where the sender P_S is registered, i.e., $P_S \in \mathcal{S}$. We first present a simplified version of our protocol for the case of exactly $t = s$ many corruptions, then explain how it can be extended to $t \leq s + \lceil (n-s)/3 \rceil - 1$ corruptions (as well as the special cases $t \in \{s+1, s+2\}$).

Recap: The Dolev-Strong Protocol. Our starting point is the classic Dolev-Strong authenticated broadcast protocol [10], which is secure against up to t Byzantine corruptions, where $t < n$. This deterministic broadcast protocol has $t + 1$ rounds and it tolerates any $t < n$ Byzantine corruptions. In each round $k \leq t + 1$, each party extends (by signing it) and forwards any valid $(k-1)$-signature chain (or simply chain, for short) that it has received in round $k - 1$, to all other parties. Here, a k-chain is defined to be valid in the view of a party P_i on a value v if the chain contains valid signatures of k different parties that do not include P_i and the first signer on the chain is the sender P_S. At the end of round $t + 1$, each party either outputs the unique value corresponding to the (only) valid signature chain it received during the course of the protocol or it outputs a default value in case such a unique value/signature chain does not exist. Validity, i.e., that for honest sender only the sender's value is received, follows immediately from unforgeability of the underlying digital signature scheme. Consistency, i.e., identical output at honest parties, follows from the fact that for rounds $k < t + 1$, a party can be sure that if it extends and forwards an accepted chain, then all other parties will either do the same in the next round, or they have already accepted it. If a party receives a $t + 1$-chain in round $t + 1$, then that chain must include at least one honest signer who would have forwarded it to all other parties in the previous round t.

The main obstacle in transferring this protocol to our setting is that if an unregistered party $P_i \in \mathcal{N}$ receives and accepts a k-chain in some round k, then it cannot extend it so that other parties will accept the extended chain in the next round. As already observed by Dolev and Strong, this issue is easy to overcome as long as $s \geq t + 1$. In this case, the protocol can proceed in $t + 1$ super-rounds which each consist of two actual rounds (referred to as *phases* below). During the first phase of super round k, an unregistered party $P_i \in \mathcal{N}$ receiving a signature chain of length k accepts it without reservation. It then forwards the chain to all registered parties in $\mathcal{S}$ in the second phase of this super round. Hence, registered parties in $\mathcal{S}$ can receive and accept chains in the second phase of a super round k and extend them in the same manner as they would in the regular Dolev-Strong protocol. This allows to carry over the same reasoning for validity and consistency to this protocol as well. Unfortunately, this approach crucially relies on there being at least one honest registered party $P_j \in \mathcal{S}$ to extend any chain that an unregistered party $P_i \in \mathcal{N}$ accepts during the protocol. Thus, for $|\mathcal{S}| \leq t$, a new insight is required.

A protocol for $s = t$. We start by adding a third phase to each super round. Moreover, we will require a total of only $s = t$ super rounds as opposed to $t + 1$ of them, as described above. We structure each super round k as follows. In the first phase, registered parties in $\mathcal{S}$ extend and forward any chain that they accepted in the previous super round (they can accept a chain during

any phase of a super round). In the second phase, unregistered parties in $\mathcal{N}$ accept and echo any k-chain received in the first phase from a registered party. Finally, in the third phase, unregistered parties accept any k-chain that they were forwarded by another unregistered party during the previous phase and once again forward it to registered parties. But an unregistered party ignores k-chains it receives during this phase of super round k. For super rounds $k < t$, the logic is relatively straight forward. If a registered party accepts a chain, every other party will accept it in the next super round $k + 1$ upon P_i extending and forwarding it during phase 1 of that super round.

If an unregistered party P_i accepts a k-chain in phase 2, then every other unregistered party P_j accepts it in phase 3 of super round k. Now, for super round t, our key insight is the following: if an unregistered party P_j accepts a t-chain in phase 3 of super round t from another unregistered party P_i, it can be certain that all parties on this chain are corrupt, as otherwise, P_j would have received this chain in phase 1 of super round t. But this implies that all unregistered parties must be honest (as there are exactly $t = s$ corruptions). Hence, P_i is honest and would have forwarded the t-chain to all other unregistered parties as well, who will reach the same conclusion. Therefore, it is safe for P_j to accept this chain from P_i.

A protocol for $t = s + \lceil (n - s)/3 \rceil - 1$ (or $t = n$ if $t \leq s + 2$). We observe that the above protocol can be viewed as a specific instantiation of a more general protocol blueprint. Namely, we can view the decision taken by unregistered parties in phase 3 of super round t as an instance of consensus on whether or not a specific chain should be accepted in this super round. We focus here on the binary case; hence assume that we simply run two separate agreement routines for each bit among the unregistered parties. It is clear that if all s registered parties are corrupted, then at most $\lceil (n - s)/3 \rceil - 1$ are corrupted among unregistered parties. This would allow unregistered parties to reach a decision on whether or not to accept a specific chain via an unauthenticated agreement protocol Π with security for up to $\lceil (n - s)/3 \rceil - 1$ many parties. Unfortunately, unregistered parties cannot know whether $\mathcal{S}$ is fully corrupted. This poses an additional issue compared to our simple protocol from above, since no guarantee on Π's behaviour can be made in case $\lceil (n - s)/3 \rceil$ or more parties are corrupted. For this reason, our construction follows a more modular and nuanced design pattern. We first build a special *gradecast* protocol in which parties additionally output a *grade* $g \in \{0, 1\}$ to indicate their confidence in their output v. If an honest party outputs v with $g = 1$, then all honest parties output v (although possibly with grade $g = 0$). In addition to the common definition of validity, graded consistency and termination [11], our protocol has a special property: it guarantees that if *any* registered party is honest, then all honest parties will output a value v with the high-confidence grade $g = 1$. Our routine follows the 3-phase super round pattern from above, but introduces some additional protocol logic to set the grades correctly. We stress that this turns out to be particularly subtle for the multivalued case (Appendix C) and constitutes the main technical challenge of our protocol.

With this gradecast in place, unregistered parties can conclude the protocol by inputting their output v from the graded consensus routine to a single instance of the unauthenticated agreement protocol Π. A party that outputs v with grade $g = 1$ in gradecast simply ignores the output of Π and outputs v as the final output of the broadcast protocol. However, all parties participate in Π so as to help parties who output in gradecast with grade 0. These low-confidence parties will then decide their output of the broadcast protocol as the output of Π. By the above discussion, if there exists at least one honest registered party, all unregistered parties ignore the output of Π and decide on a consistent output for the broadcast protocol. On the other hand, if the set of registered parties is fully corrupt, then the parties in $\mathcal{N}$ can successfully run Π. Since a single honest party with grade $g = 1$ guarantees that all parties input the same value v to Π in this case, validity of Π guarantees that all honest registered parties output consistently.

Note that this implies that t must be such that the Π has resilience $t - s$. This results in the classic resilience of strictly less than $|\mathcal{N}|/3$—except when $|\mathcal{N}| < 3$. While of course a single party can trivially solve consensus, the case of $|\mathcal{N}| = 2$ breaks the pattern due to a subtlety. Since the inputs to Π are signed by the sender, *any* signed value can be verified to be a feasible output, even a party did not receive it as input to Π. Thus, the two parties in $\mathcal{N}$ may exchange their inputs, if there is a unique value signed by the sender output it, and otherwise output a default value. We build a broadcast protocol in the Appendix B that solves this special case.

1.2 Lower Bounds

We complete our study of the partially authenticated setting with two matching lower bounds. For the case of byzantine agreement or broadcast with an unregistered sender, our lower bound follows, more or less, the standard template of impossibility proofs in this area. To illustrate the idea for the case of broadcast, our proof can leverage that the corrupted receivers can simulate the behavior of the sender with two different input bits (as it does not have a key).

Unfortunately, this strategy fails completely once the sender is registered. To illustrate our technical contribution, we describe the difficulty our proof with two strawman approaches that do not work (or only lead to a suboptimal lower bound). As our final proof mainly goes through the case of $n = 4$, $s = 1$, and $t = 2$, which it then generalizes to all other settings, our discussion below will mainly focus on this case as well. We will assume a protocol Π that solves broadcast for this setting and try to arrive at a contradiction.

Strawman 1: As we have explained, it is not possible for a corrupted party to simulate an honest sender's behaviour on two different input bits during an execution where the sender is actually honest. However, parties need to output consistently in the protocol even if the sender has crashed. For the sake of simplicity, suppose that Π is deterministic and parties output 0 after R rounds of not hearing anything from the sender. Thus, a more promising proof strategy might be to attempt to start from an execution in which the sender P_S is honest and has input 1. Two of the other parties, P_2 and P_3, are corrupted and feign, to the remaining honest party P_1, an execution in which the sender has crashed. We then move to a second execution in which the sender is dishonest, P_1 and P_2 are honest, and P_3 is corrupted. It is easy for P_S and P_3 to behave in such a way that P_1 cannot tell these first two executions apart, and so P_1 must output 1 in this second execution as well. Hence, also P_2 outputs 1 in execution 2. By a symmetrical argument, we can extend this strategy to a third execution in which P_3 and P_2 are honest so as to force P_3 to output 1 as well. However, to arrive at a contradiction, we would need an argument for why it is not possible that in this execution 1 is the output. Certainly, as the sender pretends to crash immediately, there is no way for P_3 to conclude that 1 is a valid output? Unfortunately, in order to facilitate the deception, the sender must provide signatures to the dishonest parties, so that they may "fool" P_j, $j \in \{1, 2, 3\}$ into not seeing the difference between execution j and $j + 1$. As the protocol may require the sender to sign the proposed output value 1, this overly optimistic approach fails.

Strawman 2: Having illustrated the main difficulty of this scenario, we now move to a proof strategy that is significantly closer to our actual one, except that it does not cover the case of $t \leq s < \lfloor n/4 \rfloor$. We have illustrated above why it is not possible for a corrupt receiver to simulate an honest execution on behalf of P_S with input 1 when P_S has, in actuality, benignly crashed at the beginning of the execution (and it is thus impossible to discern its input). So our key idea is instead to incrementally delay the receipt of a message in the view of an honest receiver that would be consistent with such an execution in which P_S is honest with input 1 to a later and later point in the protocol. Our strategy can achieve this while keeping the view of one honest party consistent through any two neighbouring executions while eventually pushing the receipt of a

signed message from P_S all the way back past round R. Once this has happened, we can easily switch to an execution in which P_S has actually crashed, since any party who has not received a message from P_S by round R would have had to terminate with output 0 already. Our strategy to achieve this works as follows: in the first execution, the sender is honest, whereas parties P_2 and P_3 are corrupt and feign an execution where they do not receive any messages from P_S through the first three rounds of the protocol. In the next execution, P_1 and P_2 are honest and P_S and P_3 create a view that is consistent with the first execution for P_1. Note that now, P_2 first receives a message consistent with an honest 1-execution that has originated from P_S only at round 2 (relayed through P_1). This implies that in the next step, an honest receiver P_3 will first receive such a message from P_S by round 3. By keeping P_S corrupted and rotating the remaining corruption through one of the three receivers, we can successively delay the receipt of a message that was sent from P_S to a later and later point in the protocol, as explained above.

Restrictions of Strawman 2: To make the above work, we have assumed that the protocol is deterministic and that parties can simulate the behaviour of honest protocol parties. This is possible only if parties do not keep long-term secrets on which their executions implicitly depend. As such, we restrict the scope of our proof strategy to protocols in which registered parties can only sign through an idealized functionality $\mathcal{F}_{\text{sign}}^{\mathcal{S},\mathcal{N}}$ that simultaneously allows unregistered parties to verify signatures (but not to sign them). In addition, we will assume that the adversary learns the internal state of all honest parties at the end of each round. This restricts protocol parties to rely only on short term keys and randomness. Note, however, that this still covers some very strong assumptions, like a random oracle providing unpredictable, shared randomness to all parties at the beginning of each round.

There is an additional wrinkle in this simplified strategy: it does not cover the regime where $s < \lfloor n/4 \rfloor$. To see why, we will explain the issue with translating the bound for the four party setting to the n-party setting when $s < \lfloor n/4 \rfloor$. Similar issues occur when attempting a direct proof of a bound for the n-party setting, given that $s < \lfloor n/4 \rfloor$. We assume for simplicity that all fractions in this informal discussion are integers.

The conventional template for this proof strategy is to turn an n-party protocol Π' into a 4-party protocol Π by having each of the 4 protocol parties in Π simulate $n/4$ parties in Π'. This, however, presents an immediate obstacle in the partially authenticated setting: which parties in Π' should each of the registered parties running Π simulate? Consider the following idea for the case of $s = n/4$: P_S in Π simulates P_S in Π'. All other registered parties in $\mathcal{S}$ (which are also simulated by P_S) are crashed in Π'. The remaining unregistered parties that make up $\mathcal{N}$ are divided up equally and simulated by the remaining parties $P_1, P_2, P_3 \in \mathcal{N}$ that are also running Π with P_S. Now, suppose that $s < n/4$. Hence, each of the receiver parties $P_i \in \mathcal{N}$ has to simulate $(n-s)/3 > n/4 > s$ many parties in Π'. But this poses an issue: if e.g. P_2 and P_3 become corrupted in Π (as was indeed the case for the first execution in our attack strategy described above), then there are $2(n-s)/3 > s+(n-s)/3$ many corrupted parties in Π'. But Π' is only proven secure against at most $t < s + (n-s)/3$ many corruptions. Similar issues arise for different strategies of the parties in Π simulating the parties in Π'.

Our Final Proof Strategy. Our final proof strategy follows a similar idea as above. The main technical difference is that we will start from an execution in which the sender P_S has crashed and then gradually delay the point of its crash further and further into the protocol. Meanwhile, we carefully restore the messages that an honest sender P_S would send on input 1 for those rounds where P_S is now once again sending. This argument turns out to be very delicate and requires, in some cases, to restore messages only to some of the recipients (rather than to all of them at once) and the specific round during which the sender crashes. This makes the approach work

when corrupting P_S and at most one other party P_j. This covers the entire parameter regime after translating to the general case.

Failure Probability. We note that our results show that this particular broadcast problem is structurally different. In contrast to the full-PKI or no-PKI scenarios, a sender with signing capability *can* successfully broadcast with probability arbitrarily close to 1, against any number of corruptions. This comes at the cost of a large number of rounds, however: our lower bound yields a failure probability of $\Omega(1/R)$ for any R-round protocol (subject to our model assumptions), and a simple protocol given in Section 4.3 succeeds with probability $1 - O(1/R)$.

1.3 Related Work

The Byzantine Agreement and Broadcast problems have been extensively studied in the literature, both in the computational setting [10] and in the unconditional setting [5, 14], as well as in the unconditional setting with pseudo-signatures [3, 23]. We note that Dolev and Strong [10] also considered the case where only $t + 1$ parties sign and disseminate messages. They give a Byzantine Broadcast protocol in the partially authenticated setting with s registered parties and up to $t = s - 1$ Byzantine parties.

There are several partially authenticated models that have been considered in the past literature. Specifically, Borcherding [7, 8] considers the case where signatures are only used in some rounds. We note that their setting still requires each party to hold a key. Gordon et al. [16] and Gupta et al. [17] consider a related setting with a partially compromised PKI in which the adversary is able to obtain secret keys or forge signatures on behalf of up to t_c of the honest parties, while corrupting another t_a many parties. Their work shows that broadcast is possible iff $2t_a + \min\{t_a, t_c\} < n$. Bansal et al. [2] extend the partially compromised PKI setting [16, 17] to arbitrary (undirected) graphs. A key difference compared to our setting is that in their setting, the set of parties with uncompromised keys is not publicly known, that is, any honest party's key may become exposed. As will become clear in our technical overview, our protocol crucially leverages that parties know which parties are registered and which parties are not.

Fitzi et al. [12] show that Broadcast (and multi-party computation) with unanimous abort is possible for $t < n$ corruptions in the unauthenticated setting, assuming only that secure signature schemes exist. In their protocol, parties generate keys for a signature scheme on the fly, and they use them to bootstrap a broadcast routine. We remark that their protocols would not be possible in our delayed public state model with $\mathcal{F}_{\text{sign}}^{S,N}$. This is because only signature keys that are originally registered with this functionality (and are therefore part of the partial PKI) remain secure from the adversary over the execution of the protocol. Another example of a popular cryptographic tool that falls outside of our model are verifiable random functions (VRFs). For practical purposes, VRFs [21] can be seen as a special type of signature scheme in which signatures produced through the signing algorithm have a random, close to uniform distribution. This can be used, e.g., to sample small sub-committees or justify a particular (random) choice in a protocol to another party after the fact. VRFs have been used to great effect in randomized protocols such as Algorand [15], as well as the protocols of Abraham et al. [1] and Blum et al. [6] to achieve $o(n^2)$ communication complexity.

It would be interesting to explore whether one can circumvent our lower bound by leveraging such tools and techniques (or prove a more general lower bound that rules them out, too). More generally, it would be interesting to see if or how the picture changes if protocols rely on long-term secrets beyond signing keys. For example, a common technique in multi-party computation protocols is to have parties publicly commit to some secret value v through a cryptographic commitment and later open it (if necessary) [4, 18]. We believe these are interesting directions for future work.

2 Preliminaries

2.1 Model

Network Model. We consider a complete network of n parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ and write $[n] = \{1, \ldots, n\}$ for the set of party indices. We work in the synchronous model where protocol proceeds in rounds; messages sent at the beginning of a round are guaranteed to be received by all receivers at the end of the round.

Cryptographic Setup. We consider a (bulletin) PKI setup for a subset of parties. Let $\mathcal{S} \subseteq \mathcal{P}$ denote the set of parties whose public keys are registered in the PKI and let $s := |\mathcal{S}|$. Let $\mathcal{N} := \mathcal{P} \setminus \mathcal{S}$ denote the set of unregistered parties. The registered parties in $\mathcal{S}$ have public/secret key pairs for an ideal digital signature scheme, and their corresponding public keys are known to all parties (while the secret keys are known only to the respective parties). We denote a signature of the party P_i on a message m by $\mathsf{sign}(m, i)$. For simplicity, we also refer to the tuple $(m, sign(m, i))$ as a signature on m of the party P_i.

Adversary Model. We consider an adaptive adversary that can choose up to t parties to corrupt at any time and get full control of the corrupted parties. The adversary is computationally bounded, meaning that it cannot break the security of the aforementioned signature schemes and forge signatures. We note that our lower bounds even rule out protocols in the presence of a static adversary, which makes the lower bounds even stronger.

2.2 Definitions

Definition 2.1 (Byzantine Agreement (BA)). Let Π be a protocol executed among parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ where each party $P_i \in \mathcal{P}$ starts with an input value v_i and outputs a value o_i upon termination. Π solves BA against an adversary corrupting up to t parties if it satisfies the following properties:

- **Termination:** Every honest party P_i terminates.
- **Agreement:** For any two honest parties $P_i, P_j \in \mathcal{P}$ with outputs o_i and o_j, respectively, it holds that $o_i = o_j$.
- **Validity:** If each honest party P_i starts with the same input value v, then every honest party outputs $o_i = v$.

Definition 2.2 (Byzantine Broadcast). Let Π be a protocol executed among parties $\mathcal{P} = \{P_1, \ldots, P_n\}$. Denote the designated sender by P_S. In a broadcast protocol, P_S starts with an input value v and each party $P_i \in \mathcal{P}$ outputs a value v_i upon terminating. Π solves broadcast against an adversary corrupting up to t parties if it satisfies the following properties:

- **Termination:** Every honest party terminates.
- **Consistency:** For any two honest parties $P_i, P_j \in \mathcal{P}$, it holds that $v_i = v_j$.
- **Validity:** If the sender P_S is honest with input v, then every honest party outputs $v_i = v$.

Definition 2.3 (Gradecast). Let Π be a protocol executed among parties $\mathcal{P} = \{P_1, \ldots, P_n\}$. Denote the designated sender by P_S. P_S starts with an input value v, and each party $P_i \in \mathcal{P}$ outputs a value v_i upon terminating. Π solves gradecast against an adversary corrupting up to t parties if it satisfies the following properties:

- **Termination:** Every honest party terminates.
- **Graded Consistency:** If some honest party $P_i \in \mathcal{P}$ outputs (v_i, g_i) with $g_i = 1$, it holds that $v_i = v_j$ for any honest party $P_j \in \mathcal{P}$ that outputs (v_j, g_j).
- **Validity:** If the sender P_S is honest and it starts with input v, then every honest party outputs (v_i, g_i) such that $v_i = v$ and $g_i = 1$.

3 Broadcast with a Registered Sender in the Partially Authenticated Setting

In this section, we present the main positive result of our work: a broadcast protocol with a registered sender $P_S \in \mathcal{S}$ and resilience $t = s + \lceil (n-s)/3 \rceil - 1$ (or $t = n$ if $t \leq s + 2$). We focus here on the main case of interest where $n - s \geq 3$ and cover the case of $n - s < 3$ in Appendix B. A construction for the multivalued case is given in Appendix C.

3.1 Gradecast in the Partially Authenticated Setting

This subsection introduces the main primitive behind our broadcast protocol: a (binary) *gradecast* protocol Π_{GC} in the partially authenticated setting described in Fig. 1. Informally, gradecast [11] is a weakened form of broadcast in that each party additionally outputs a *grade* reflecting its confidence. Concretely, at the end of Π_{GC}, each honest party P_i outputs a pair (v_i, g_i), where $v_i \in \{0, 1\}$ is its tentative decision for the sender's bit and $g_i \in \{0, 1\}$ is a grade indicating whether P_i believes that agreement on v_i holds for all honest parties. Our validity and graded consistency matches the standard gradecast definition [19]. Our Π_{GC} protocol also achieves a special conditional grade-1 property, that if there exists an honest registered party, all honest parties output with grade 1; we will show this in Lemma 3.9. Crucially, Π_{GC} works for any number of corruptions. We first recall the signature chains. Similar as in the classic Dolev-Strong authenticated broadcast protocol [10],

Definition 3.1 (Signature Chains). A *k-signature chain* C_k (for $k > 0$) with sender P_S on value v is recursively defined as follows: a 1-signature chain on v is a pair $(v, \mathsf{sign}(v, S))$. For $k > 1$, a k-signature chain on v is a pair $(C_{k-1}, \mathsf{sign}(C_{k-1}, i))$, where C_{k-1} is a $(k-1)$-signature chain on v that does not contain a signature from P_i. A k-signature chain on a value v is *valid* for P_i if the following conditions are satisfied:

- C_k begins with a signature of P_S on the value v.
- All k signers are distinct.
- C_k does not contain a signature of P_i (this trivially holds for $P_i \in \mathcal{N}$).

We first prove several useful lemmas for our protocol Π_{GC} that tolerates t Byzantine parties with resilience $t < n$.

Lemma 3.2. *There is no registered party $P_i \in \mathcal{S}$ that sets $\text{acc}_i^s = 1$ in the super round s.*

Proof. P_i only accepts an s-chain C_s if C_s contains signatures from s different parties and it does not contain the signature of P_i. Since there exist at most s parties that can sign on the chain, there exists no such C_s that P_i can accept in the super round s. □

Lemma 3.3. *If there is an honest registered party $P_i \in \mathcal{S}$ that outputs (v_i, g_i) such that $v_i = 1$, then $v_j = 1$ for any honest party P_j that outputs (v_j, g_j).*

Proof. Since each registered party $P_i \in \mathcal{S}$ outputs v_i only if it has accepted a k-chain in super round k, from Lemma 3.2, the accepted chain must be a k-chain for $k < s$. Therefore, P_i creates a valid $k+1$-chain C_{k+1} and sends $(C_{k+1}, \mathsf{Extension})$ to all parties in the phase 1 in super round $k+1$. Each party P_j with $v_j = 0$ sets $v_j = 1$ in the phase 2 of super round $k+1$. Furthermore, P_j never sets $v_j = 0$ once it has $v_j = 1$ according to the protocol, so P_j must output $v_j = 1$ at the end of the protocol Π_{GC}. □

Lemma 3.4. *If there is an honest registered party $P_i \in \mathcal{S}$ that outputs (v_i, g_i) such that $v_i = 0$, then $v_j = 0$ for any honest party P_j that outputs (v_j, g_j).*

Proof. Assume $P_i \in \mathcal{S}$ outputs (v_i, g_i) such that $v_i = 0$. This means it has not accepted any chain during any super round of the protocol. Toward a contradiction, let P_j be an honest party

(Binary) Gradecast Protocol Π_{GC}

Input and Initialization. Denote the Sender $P_S \in \mathcal{S}$. Each party P_i sets its grade $g_i = 1$ and value $v_i = 0$ at the beginning of the protocol. Each registered party P_i initializes a flag $\mathsf{acc}_i^k = 0$ to indicate whether P_i has accepted a k-chain in super round k, for $k < s$. If $P_i = P_S$ and $b = 1$, set $v_i := 1$ and $\mathsf{acc}_i^0 = 1$ (treated as accepting a valid 0-chain).

For $k = 1, \ldots, s$, execute super round k as described below:

Phase 1: PKI Extension. Each registered party P_i: if $\mathsf{acc}_i^{k-1} = 1$ (which means P_i has accepted a valid $(k-1)$-chain that does not contain the signature of P_i in super round $k-1$), create k-chain $C_k = \mathsf{sign}(C_{k-1}, i)$ and send Extension message $(C_k, \mathsf{Extension})$ to all parties.

Phase 2: Echo from Non-PKI Parties.

- Each registered party P_i that received a valid k-chain C_k in an Extension message $(C_k, \mathsf{Extension})$ in Phase 1: if $v_i = 0$, set $v_i = 1$ and $\mathsf{acc}_i^k = 1$.
- Each unregistered party P_i that received a valid k-chain C_k in an Extension message $(C_k, \mathsf{Extension})$ in Phase 1:
  - if $v_i = 0$, set $v_i = 1$ and send Echo_1 message (C_k, Echo_1) to all parties.
  - if $v_i = 1$ and $g_i = 0$, set $g_i = 1$ and send Echo_1 message (C_k, Echo_1) to all parties.

Phase 3: Chain Acceptance from $\mathcal{N}$.

(1)
- Each registered party P_i: if $v_i = 0$ and P_i received a valid k-chain C_k in an Echo_1 message (C_k, Echo_1) in Phase 2 set $v_i = 1$ and $\mathsf{acc}_i^k = 1$.
- Each unregistered party P_i: if $v_i = 0$ and it received a valid k-chain in an Echo_1 message (C_k, Echo_1) in Phase 2, set $v_i = 1$, $g_i = 0$ and send (C_k, Echo_2) to all parties.

(2)
- Each registered party P_i: if $v_i = 0$ and it received a valid k-chain C_k in an Echo_2 message in the last round, set $\mathsf{acc}_i^k = 1$ and $v_i = 1$.
- Each unregistered party P_i: if it received a valid k-chain C_k in an Echo_2 message (C_k, Echo_2) in the last round and $v_i = 0$: set $g_i = 0$ and send Echo_3 message (C_k, Echo_3) to all registered parties.

At the end of Phase 3: For each registered party P_i: if it received a valid k-chain C_k in an Echo_3 message (C_k, Echo_3) in the last round and $v_i = 0$, set $\mathsf{acc}_i^k = 1$ and $v_i = 1$.

Output Rule. At the end of super round s, for each party P_i: output (v_i, g_i).

Fig. 1. (Binary) Gradecast Protocol Π_{GC}

that sets $v_j = 1$. We consider two cases: first, if P_j is registered, then by Lemma 3.3, P_i must output $v_i = 1$ at the end of the protocol, contradicting to the assumption that $v_i = 0$. In the second case, we have P_j to be unregistered. Then, P_j sets $v_j = 1$ in super round k for some $k < s$. Since the k-chain C_k that P_j received cannot contain the signature of P_i (as we have assumed that it never accepts a chain in any super round), $k < s$. There are the following sub-cases:

- If $P_j \in \mathcal{N}$ sets $v_j = 1$ in phase 2 of super round k, P_i must receive the Echo_1 message containing C_k of P_j in phase 2 of super round k. P_i sets $v_i = 1$ and outputs (v_i, g_i) with $v_i = 1$ at the end of the protocol.
- If $P_j \in \mathcal{N}$ sets $v_j = 1$ in phase 3 of super round k, P_i must receive the chain either from an Echo_2 message or Echo_3 message from P_j in phase 3 of super round k. Upon receiving the k-chain C_k as part of one of those messages, P_i sets $v_i = 1$ and outputs (v_i, g_i) with $v_i = 1$ at the end of the protocol.

Clearly, both of these cases lead to a contradiction with P_i outputting $(v_i = 0, g_i)$. Therefore, in conclusion, each party P_j must output (v_i, g_i) such that $v_i = 0$ at the end of the protocol. □

Lemma 3.5. *If there is an honest registered party $P_i \in \mathcal{S}$ that outputs (v_i, g_i), each honest party P_j outputs (v_j, g_j) with $v_j = v_i$.*

Proof. The proof follows directly from Lemma 3.3 and Lemma 3.4. □

Lemma 3.6. *The Gradecast protocol Π_{GC} satisfies Validity.*

Proof. If the sender P_S is honest and it starts with input 1, each honest party P_i sets $v_i = 1$ in phase 2 of super round 1 upon receiving a 1-chain from P_S. Hence, all honest parties in $\mathcal{N}$ must output $(1, 1)$ at the end of the protocol. Each honest party $P_j \in \mathcal{S}$ receives the Extension message from the leader P_S in, they must set $v_i = 1$ in the phase 2 and output $(1, 1)$ at the end of the protocol. If the sender P_S is honest and it starts with input 0, we note that in our protocol, the leader does not accept any valid 0-chain and it will not set $v_S := 1$ if $b = 0$. As a result, no honest party accepts any chain during the protocol from the unforgeability of the ideal signature scheme. In conclusion, any honest party must output $(0, 1)$ at the end of the protocol. □

Lemma 3.7. *The protocol Π_{GC} satisfies Graded Consistency.*

Proof. Assume an honest party P_i outputs (v_i, g_i) such that $g_i = 1$. If there exists an honest party P_j (P_j could be P_i) $\in \mathcal{S}$, from Lemma 3.5, all honest parties must output the same value as v_j. So assume instead that all honest parties are in the set $\mathcal{N}$. There are following different cases:

- **Case 1.** $v_i = 1$**.** In this case, $g_i = 1$ only happens if P_i sets $v_i = 1$ in phase 2 in a super round k. P_i will send an Echo_1 message to all other parties in phase 2 of super round k. Thus, each party $P_j \in \mathcal{N}$ sets $v_j = 1$ in phase 3 of super round k and output (v_j, g_j) such that $v_j = 1$.
- **Case 2.** $v_i = 0$**.** In this case, $g_i = 1$ only happens if there is no honest party P_j that sets $v_j = 1$ in any super round $k < s$. Now consider the last super round s. If any honest party $P_j \in \mathcal{N}$ sets $v_j = 1$ in phase 2 or the first round of phase 3, P_j either sends an Echo_1 message or an Echo_2 message to P_i. Thus, P_i would set $v_i = 1$ in phase 3, which is a contradiction. As a result, each honest party P_j must output (v_j, g_j) such that $v_j = 0$ at the end of the protocol.

In conclusion, the protocol Π_{GC} satisfies Graded Consistency. □

Lemma 3.8. *The protocol Π_{GC} satisfies Termination.*

Proof. Each party P_i in the protocol Π_{GC} terminates and outputs (v_i, g_i) after s super rounds where each super rounds consist of 4 rounds. □

Moreover, our Gradecast protocol Π_{GC} has a specialized grade-1 property in the partially authenticated setting, that if there exists an honest registered party $P_i \in \mathcal{S}$, each honest party P_j outputs (v_j, g_j) with grade $g_j = 1$ at the end of the protocol Π_{GC}:

Lemma 3.9. *If there exists an honest registered party $P_i \in \mathcal{S}$ (including the leader P_S), each honest party P_h outputs (v_h, g_h) such that $g_h = 1$ at the end of the protocol Π_{GC}.*

Proof. Assume an honest party $P_i \in \mathcal{S}$ outputs (v_i, g_i) at the end of the protocol. Throughout the protocol, $P_i \in \mathcal{S}$ only outputs with grade $g_i = 1$. Now we distinguish two cases for the value v_i and an arbitrary honest party $P_j \in \mathcal{N}$:

- If $v_i = 1$, from Lemma 3.2, P_i only accepts a k-chain and sets $v_i = 1$ in super round $k < s$. Since P_i only accepts a chain which it isn't itself a part of, it can extend the chain by adding its own signature to it and send an Extension message $(C_{k+1}, \text{Extension})$ for the extended $(k+1)$-chain C_{k+1} in super round $k+1$. Thus, each honest party P_j sets $v_j = 1$ and it must have $g_j = 1$ in phase 2 of super round $k+1$.

- If $v_i = 0$, assume there is an honest party $P_j \in \mathcal{N}$ that outputs (v_j, g_j) with $g_j = 0$. Since $g_i = 1$, by graded consistency (Lemma 3.7), P_j must output $v_j = v_i = 0$. P_j only sets $g_j = 0$ in phase 3 of some super round $k \leq s$, upon receiving a valid k-chain C_k in an Echo_2 message. If $k = s$, the chain must contain the signature of P_i. Thus $v_i = 1$, contradicting the assumption that $v_i = 0$. If $k < s$, P_j sends an Echo_3 message containing the k-chain C_k to every party in $\mathcal{S}$, including P_i. Therefore, P_i sets $v_i = 1$ at the end of Phase 3 of super round $k < s$, contradicting the assumption that $v_i = 0$. □

3.2 Broadcast in the Partially Authenticated Setting

We are now ready to introduce our broadcast protocol Π_{BC} that builds on top of Π_{GC}, assuming a deterministic unauthenticated BA protocol Π_{unau} (with inputs that come from the outputs of Π_{GC}) among $n - s$ unregistered parties that tolerates up to $\lceil (n-s)/3 \rceil - 1$ Byzantine parties (for example, the one in [5]). In particular, for each party P_i that outputs (v_i, g_i) in the Π_{GC} protocol with $g_i = 1$, P_i outputs v_i at the end of Π_{BC}. Each unregistered party $P_i \in \mathcal{N}$ inputs its value v_i to the deterministic unauthenticated BA protocol Π_{unau}, denote its output as m_i. If $g_i = 1$, P_i outputs v_i, if $g_i = 0$, P_i outputs m_i. We note that only unregistered parties may output with grade 0 from our description of the Π_{GC} protocol.

Broadcast Protocol Π_{BC}

Input and Initialization. Denote the sender $P_S \in \mathcal{S}$. Each party P_i sets value $v_i = 0$ at the beginning of the protocol. If $P_i = P_S$ and $b = 1$, set $v_i \coloneqq 1$.

Gradecast. Each party P_i: participate in the Gradecast protocol Π_{GC} with sender P_S and its input b.

Denote the output of party P_i to be (v_i, g_i).

Consensus of Non-PKI parties.

- Each unregistered party $P_i \in \mathcal{N}$: input v_i to the deterministic unauthenticated BA protocol Π_{unau}, denote the output of P_i to be m_i.
- Each registered party P_i in $\mathcal{S}$: do not participate in the deterministic unauthenticated BA and wait for it to terminate; we use a fixed-round unauthenticated BA (e.g., the one in [5], which takes $3(\lceil (n-s)/3 \rceil - 1) + 1$ rounds).

Output Rule.

- Each registered party $P_i \in \mathcal{S}$: output v_i.
- Each unregistered party $P_j \in \mathcal{N}$:
  - If $g_j = 1$, output v_j.
  - Otherwise, output m_j.

Fig. 2. Byzantine Broadcast Protocol Π_{BC} in the partially authenticated setting

Lemma 3.10. *The protocol Π_{BC} satisfies Validity if $t \leq s + \lceil (n-s)/3 \rceil - 1$.*

Proof. Assume the sender is honest and starts with input b. According to the validity of the protocol Π_{GC}, each honest party P_j must output $(v_j, g_j) = (b, 1)$ at the end of the protocol. According to the output rule of the protocol Π_{BC}, for each honest party $P_i \in \mathcal{S}$, P_i must output $v_i = 1$. For each honest party $P_j \in \mathcal{N}$, we make a case distinction:

- **Case 1.** $b = 0$. Since $v_j = 0$ and $g_j = 1$, P_j must output 0 according to the output rule of Π_{BC}.
- **Case 2.** $b = 1$. We have $v_j = 1$ and $g_j = 1$, none of the two conditions in the output rule are satisfied, and the party P_j must outputs 1.

In conclusion, each honest party outputs b. □

Lemma 3.11. *The protocol Π_{BC} satisfies Consistency if $t \leq s + \lceil (n-s)/3 \rceil - 1$.*

Proof. We consider two cases:

- **Case 1. There exists an honest party $P_i \in \mathcal{S}$.** In this case, denote its output of Π_{GC} as (v_i, g_i). From special grade-1 property (Lemma 3.9), each honest party P_h outputs (v_h, g_h) such that $g_h = 1$ at the end of the protocol Π_{GC}. From the graded consistency, each honest party P_h must output $v_h = v_i$. According to the output rule, each party $P_x \in \mathcal{S}$ therefore outputs $v_x = v_i$ and outputs v_i, and each party P_j in $\mathcal{N}$ has $g_j = 1$, so it outputs v_i from our output rule. This means that each honest party P_j outputs v_i, finishing this case.
- **Case 2. All parties in $\mathcal{S}$ are Byzantine.** Since $t \leq s + \lceil (n-s)/3 \rceil - 1$, we have $3(t-s) \leq 3\lceil (n-s)/3 \rceil - 3 < n - s$. If all parties in $\mathcal{S}$ are Byzantine, then there are at most $t - s$ Byzantine parties in $\mathcal{N}$, which means that the deterministic unauthenticated BA among parties in $\mathcal{N}$ must be secure. As a result, all honest parties in the set $\mathcal{N}$ must have the same output m for the unauthenticated BA.
  - **Case 2a. There exists an honest party $P_j \in \mathcal{N}$ that outputs (v_j, g_j) with $g_j = 1$.** In this case, from the graded consistency (Lemma 3.7), each honest party P_h must output (v_h, g_h) in the Gradecast protocol Π_{GC} such that $v_h = v_j$. From the validity of the unauthenticated BA, all honest parties output the same value m at the end of Π_{BC}.
  - **Case 2b. All honest parties have grade 0.** In this case, their output value is fully determined by their outputs of the unauthenticated BA, according to the output rule. As we have already established that all honest parties have the same output m in the unauthenticated BA protocol, consistency follows in this case.

□

Lemma 3.12. *The protocol Π_{BC} terminates in a finite number of rounds if $t \leq s + \lceil (n-s)/3 \rceil - 1$.*

Proof. This follows from the fact that Gradecast Π_{GC} satisfies termination, and the unauthenticated BA [5] runs for $3(\lceil (n-s)/3 \rceil) + 1$ rounds (we note that honest parties terminate after $3(\lceil (n-s)/3 \rceil) + 1$ rounds according to the protocol even if there exists more Byzantine parties in the protocol). □

Theorem 3.13. *The Protocol Π_{BC} is a Byzantine Broadcast Protocol that tolerates $t \leq s + \lceil (n-s)/3 \rceil - 1$ Byzantine parties, given the condition that s parties (including the sender) have PKI setups.*

Proof. This follows directly from Lemma 3.10, Lemma 3.11 and Lemma 3.12. □

Interestingly, in the special case when $n - s < 3$, we can even tolerate $t \leq s + 1$ corruptions. We state the result here and leave proofs as well as protocol Π_{BCS} constructions (Fig. 4) in our Appendix B.

Theorem 3.14. *If $n - s < 3$, there exists a Byzantine Broadcast protocol that tolerates any number of corruptions.*

Theorem 3.15. *The communication complexity of the Binary Byzantine Broadcast protocol Π_{BC} in Figure. 2 is $O(s \cdot n^2) + O((n-s)^3)$.*

Proof. In the binary broadcast protocol Π_{BC}, each registered party $P_i \in \mathcal{S}$ forwards chains only upon accepting a chain and does not send a second Extension message. Each unregistered party $P_i \in \mathcal{N}$ sends at most two chain-carrying messages, only when its value changes ($v_i : 0 \to 1$) or when its grade changes ($g_i : 0 \to 1$). Each chain has length of at most s. All unregistered parties invoke the unauthenticated BA in [5] once, which costs $O((n-s)^3)$. Overall, the communication complexity is $O(s \cdot n^2) + O((n-s)^3)$. □

4 Lower Bounds

4.1 Lower bounds for BA in the Partially Authenticated Setting

We now show that the resilience bound for Byzantine Agreement

$$t \leq \max\left\{\lceil s/2 \rceil,\ \lceil n/3 \rceil\right\} - 1$$

is indeed optimal. In particular, when $s = 0$, i.e., no PKI is available, it is well-known that $t \leq \lceil n/3 \rceil - 1$ is the tight bound. When $s = n$, i.e., all parties are registered with PKI setup, it is also well-known that $t \leq \lceil s/2 \rceil - 1$ is the tight bound. We first prove it is optimal for broadcast with respect to an unregistered sender (recall that in the Corollary A.2, we show there exists a Broadcast protocol that tolerates up to $t \leq \max\left\{\lceil s/2 \rceil,\ \lceil n/3 \rceil\right\} - 1$ Byzantine parties, if the sender is unregistered).

Theorem 4.1. *Assume there exist s registered parties and up to t parties may be Byzantine. Then there is no Byzantine Broadcast protocol if*

$$t \geq \max\left\{\lceil s/2 \rceil,\ \lceil n/3 \rceil\right\}.$$

Proof. Suppose that Π is a protocol for Byzantine Broadcast which is secure against

$$t = \max\left\{\lceil s/2 \rceil,\ \lceil n/3 \rceil\right\}.$$

Toward a contradiction, we divide parties into 3 groups: $A \subset \mathcal{S}$ and $C \subset \mathcal{S}$ are disjoint sets of size $\max\left\{\lceil s/2 \rceil,\ \lceil n/3 \rceil\right\}$, whereas $B \subset \mathcal{N}$ is a group consisting of the remaining $n - |A| - |C| \leq n - s$ unkeyed parties (including the sender P_S). We note that, $n - |A| - |C| \leq n - 2\lceil n/3 \rceil \leq \lceil n/3 \rceil \leq \max\left\{\lceil s/2 \rceil,\ \lceil n/3 \rceil\right\}$, and so the adversary has sufficient budget to fully corrupt any one of these sets. Moreover, these sets are well-defined since $2 \cdot \lceil n/3 \rceil \geq |A| + |C| \geq s$, and so B can be chosen to be both non-empty and as a subset of non-registered parties ($\mathcal{N}$). Crucially, this allows the adversary to simulate the behaviour of parties in B. We prove a contradiction via the following three executions:

(1) In the first execution, P_S in B starts with input 1, all parties in A and B are honest. All parties in C are Byzantine and simulate toward A an execution in which parties in C are honest, but group B is communicating to parties in C as though P_S starts with input 0. They behave honestly toward B. According to the validity requirement, all (honest) parties in groups A and B must output 1 at the end of the protocol.
(2) In the second execution, P_S in B starts with input 0. All parties in B and C are honest, all parties in A are Byzantine. Parties in A simulate toward the group C an execution in which group B is communicating to parties in A as though P_S starts with input 0. According to the validity requirement, all parties in group C must output 0 at the end of the protocol.
(3) In the third execution, all parties in A and C are honest. All parties in B (including P_S) are Byzantine and they simulate an execution to A in which P_S starts with input 1. They simulate toward C an execution in which P_S starts with input 0. Since the view of parties in A is the same as in execution 1, all parties in A must output 1. Since the view of parties in C is the same as in execution 2, all parties in C must output 0. In conclusion, consistency is violated.

□

Corollary 4.2. *Assume there exists s registered parties and t parties can be Byzantine. Then there is no Byzantine Agreement protocol if*

$$t \geq \max\left\{\lceil s/2 \rceil, \lceil n/3 \rceil\right\}.$$

Proof. Assume there exists a BA protocol in the partially authenticated setting with s registered parties that tolerates up to t Byzantine parties, such that $t \geq \max\left\{\lceil s/2 \rceil, \lceil n/3 \rceil\right\}$, we can design a BC protocol as follows: the sender P_S first sends its input to all other parties, all parties run this BA protocol with input as the value received from P_S and output its BA output. Since there exists no such BC protocol, there cannot exist BA protocol with resilience

$$t \geq \max\left\{\lceil s/2 \rceil, \lceil n/3 \rceil\right\}.$$

□

4.2 Lower bounds for BC in the Partially Authenticated Setting with Public State

We now give a lower bound for broadcast protocols in the partially authenticated setting. The bound holds in a model in which registered parties can securely use digital signatures, but otherwise all state of parties is known to the adversary, and consequently using digital signatures and related cryptographic primitives does not help parties.

The lower bound works in the same model as discussed in Section 2, with the following two modifications:

Delayed Public State Model. The adversary can base its actions in round r on the full state of all parties at the end of the preceding round $r-1$ (or the initial state, if $r = 1$). More precisely, at the end of each round, the full internal state[2] of each party is leaked to the adversary. Parties may generate and use any kind of private or shared randomness. However, any randomness generated in round r is leaked to the adversary at the end of round r as well. Thus, the adversary must choose which messages corrupted parties send in round r without knowledge of the private randomness honest parties generate in round r, but gains complete information about round r once it ends.

Partial Signing Functionality. All parties (and the adversary) have access to a special ideal functionality $\mathcal{F}_{\text{sign}}^{\mathcal{S},\mathcal{N}}$ parameterized by the sets $\mathcal{S}, \mathcal{N}$ (see Section 2). The ideal functionality internally maintains an initially empty set Auth and offers the following interfaces:

- $\mathcal{F}_{\text{sign}}^{\mathcal{S},\mathcal{N}}.\texttt{Sign}(m)$, called by Party $i \in \mathcal{S}$: Set Auth := Auth $\cup \{(i, m)\}$.
- $\mathcal{F}_{\text{sign}}^{\mathcal{S},\mathcal{N}}.\texttt{Verify}(i, m)$, called by Party $j \in \mathcal{S} \cup \mathcal{N}$: Return 1 if $(i, m) \in$ Auth. Otherwise, return 0.

Notably, unregistered parties in $\mathcal{N}$ cannot call $\mathcal{F}_{\text{sign}}^{\mathcal{S},\mathcal{N}}.\texttt{Sign}(m)$. Intuitively, this models that unregistered parties cannot use digital signatures (in a meaningful way): while they could still sign with a digital signature scheme without calling the ideal functionality, their secret key would be accessible by the adversary as part of their state.

Remark. The way we defined the signature functionality implies that we consider protocols in which signatures are not used in any other way than for signing and verification. For instance, the model does not allow parties to encrypt or hash signatures. For our purposes, this minimalistic ideal functionality is sufficient, and we refer to [9] for a discussion on ideal functionalities for digital signatures.

[2]When modeling parties as interactive Turing machines, the full internal state consists of all tapes, the position of the head, its state transition function, and its state register.

Theorem 4.3. *Suppose that $t - s \geq (n - s)/3$ with $n - s \geq 3$. Then there is no broadcast protocol that terminates in R rounds and succeeds with probability $1 - 1/(8R)$, even for a registered sender, according to the model defined above.*

Proof. We first consider the special case that $n = 4$, $s = 1$, and the sender and up to one other party can be dishonest (i.e., $t = 2$). Denote the sender by P_S and the other parties by P_j, $j \in \{1, 2, 3\}$. Assume for contradiction that a protocol Π violating the statement of the theorem exists for this setting.

For all $1 \leq r \leq R$, we define an ensemble of partial executions $\mathcal{E}^r = (\mathcal{E}_i^r)_{1 \leq i \leq 4R}$ of Π. For convenience, we set, for all $1 \leq i \leq 4R$, $\mathcal{E}_i := \mathcal{E}_i^R$. We note that we are mainly interested in these executions. The main purpose of defining the partial executions $\mathcal{E}_i^r$ for $r < R$ was to give a proper definition of $\mathcal{E}_i^R$.

For all r and all executions $\mathcal{E}_i^r \in \mathcal{E}^r$, the party $P_{i \bmod 3}$ is Byzantine. This party equivocates towards the two other parties P_j with $j \neq i \bmod 3$ so that one cannot distinguish $\mathcal{E}_i$ from $\mathcal{E}_{i-1}$ and the other not from $\mathcal{E}_{i+1}$; we discuss how this is achieved below.

The random coins ρ_1, ρ_2, ρ_3 for the honest parties are sampled once and then fixed to the same value across all of the collective $4R^2$ many partial executions in these ensembles. The sender P_S behaves like a crashing honest party with some arbitrary, but fixed input x and random coins ρ_S which are also fixed to the same value for all of the partial executions. In each partial execution $\mathcal{E}_i$, it "crashes" in such a way that its message to party $P_{j \bmod 3}$ in round r is delivered if and only if $r \leq (i - j - R)/3$. That is, P_S simulates honest behavior up to and including round

$$r_i := \max\left\{r \in \mathbb{N} \mid r \leq \frac{i - 3 - R}{3}\right\} = \max\left\{\left\lfloor \frac{i - R}{3} \right\rfloor - 1, 0\right\},$$

and sends to $P_{j \bmod 3}$ in round $r_i + 1$ the message it would send if it was honest if and only if $i > R$ and $j \leq i \bmod 3$.

Intuitively, this pattern (i) connects the fully honest behavior of the sender in $\mathcal{E}_{4R}$ (where the output must hence be x) to $\mathcal{E}_1$, in which honest parties never receive any signature (and hence must not output x), while (ii) always crashing P_S before it would enable the honest parties in $\mathcal{E}_i$ to distinguish the execution from $\mathcal{E}_{i-1}$ or $\mathcal{E}_{i+1}$, respectively.

Execution definition. We now give the more precise definition of $\mathcal{E}^r$. $\mathcal{E}^r$ is defined inductively over r as described in the following. For the base case $r = 0$, we define all $\mathcal{E}_i^0 \in \mathcal{E}^0$ as follows. The execution of Π is halted before the onset of the first round, i.e., parties send no messages in Π. It is clear that these partial executions are determined entirely by the initial states of the honest parties. For the inductive case $r \geq 1$: By the induction hypothesis, the first $r - 1$ rounds of execution $\mathcal{E}_i^r$ have already been constructed as $\mathcal{E}_i^{r-1}$; we need to construct round r of each execution $\mathcal{E}_i^r$. The execution of Π is halted immediately after the rth round, i.e., parties send messages only during the first r rounds of Π in $\mathcal{E}_i^r$.

For $1 \leq i < 4R - 1$, we say that $P_{i \bmod 3}$ $(i + 1)$-*simulates* messages in Execution $\mathcal{E}_i^r$ if $P_{i \bmod 3}$ does the following. First, it simulates the partial execution $\mathcal{E}_{i+1}^{r-1}$, which by definition halts after round $r - 1$. Then, it simulates how $P_{i \bmod 3}$ would honestly derive its messages for round r based on its simulated state after the conclusion of round $r - 1$ in this simulation of $\mathcal{E}_{i+1}^{r-1}$. For the special case of $\mathcal{E}_{4R}^r$, $P_{4R \bmod 3}$ $(4R + 1)$-simulates messages toward $P_{4R-1 \bmod 3}$ by emulating the same behavior as the honest $P_{4R \bmod 3}$ has in $\mathcal{E}_{4R-1}^r$. We define $P_{i \bmod 3}$'s $(i - 1)$-*simulation* of messages in $\mathcal{E}_i^r$ for $1 \leq i < 4R$ in the analogous manner. In execution $\mathcal{E}_1^r$, P_1's 0-*simulation* is by emulating the same behavior as the honest P_1 in $\mathcal{E}_2^r$.

Equipped with this terminology, we now specify the messages sent by dishonest parties in round r of $\mathcal{E}_i^r$ for all $1 \leq i \leq 4R$.

- $P_{i \bmod 3}$ $(i+1)$-simulates its messages to $P_{i-1 \bmod 3}$ and P_S and $(i-1)$-simulates its message to $P_{i+1 \bmod 3}$.
- If $r \leq r_i$, the sender P_S transmits messages according to its simulation of an honest party on input x and fixed random coins ρ_S.
- If $r = r_i + 1$, the sender P_S transmits messages according to its simulation of an honest party on input x to each party P_j for which $r_i + 1 \leq (i - j - R)/3$. Otherwise, the sender transmits no message to P_j.
- If $r > r_i + 1$, the sender sends no messages in round r.

Note that the above specification is feasible in our system model if and only if the dishonest parties can carry out the above simulation. Given that all executions use the same randomness for all parties, which is revealed at the end of each round, and the initial states are identical in all executions, this is immediate provided the adversary has access to all signatures created by P_S in all executions $\mathcal{E}_i^{r-1}$. Given that P_S is dishonest, the adversary has direct access to the signing functionality, so this is trivially satisfied.

However, it is essential to prove that this is not necessary in $\mathcal{E}_{4R}$. In this execution, P_S sends exactly the messages it would send if being honest. If it also accesses the signing functionality as specified by Π, it *is* honest in this execution, forcing honest parties to output x by the validity property. This is a key step in deriving a contradiction to the existence of Π, as in $\mathcal{E}_1$ (dishonest) P_S never signs any message, preventing any non-default output. For convenience, we define $\mathcal{E}_i(x)$ as the execution $\mathcal{E}_i$ in which the input of the sender is fixed to x.

Lemma 4.4. *For input values $x \neq y$, honest parties cannot distinguish $\mathcal{E}_1(x)$ and $\mathcal{E}_1(y)$.*

Proof. We prove by induction on r the more general statement for $0 \leq r \leq R$ and $1 \leq i \leq R-r+1$, honest parties cannot distinguish $\mathcal{E}_i^r(x)$ and $\mathcal{E}_i^r(y)$. As the algorithm terminates by the end of round R, for $r = R$ and $i = 1$ this yields the claim of the lemma. The induction is anchored at $r = 0$, where the claim trivially holds for all i, since all parties use the same randomness in all executions and only the dishonest party P_S receives any input.

For the step from $0 \leq r - 1 \leq R - 1$ to r, by the induction hypothesis and the fact that in all executions the randomness of each party is the same, all honest parties send the same messages in round r of each considered execution. Note that P_S sends no messages in any of them and the induction hypothesis guarantees indistinguishability throughout the first $r - 1$ rounds. Therefore, the only possibility for honest party P_j to distinguish $\mathcal{E}_i^r(x)$ and $\mathcal{E}_i^r(y)$ is by the message received from dishonest party $P_{i \bmod 3}$ in round r of these executions. However, depending on the recipient, $P_{i \bmod 3}$ sends the same message as in one of the executions $\mathcal{E}_{i+1}^{r-1}(x)$, $\mathcal{E}_{i+1}^{r-1}r(y)$, $\mathcal{E}_{i-1}^{r-1}(x)$, or $\mathcal{E}_{i-1}^{r-1}(y)$. By the induction hypothesis, these are all indistinguishable to $P_{i \bmod 3}$, i.e., the message sent to P_j is the same in all cases. □

Corollary 4.5. *There is an input value x with the following property. If all parties' randomness is sampled according to the distribution specified for honest parties, the probability that all honest parties output x in execution $\mathcal{E}_1(x)$ is at most $1/2$.*

Proof. By Lemma 4.4, for fixed randomness honest parties cannot distinguish $\mathcal{E}_1(x)$ and $\mathcal{E}_1(y)$ for $x \neq y$. Thus, their output is a function of the randomness only. As honest party P_j outputs only a single value o_j in each execution, we have that $P[o_j = x] + P[o_j = y] \leq 1$ (where the probability is taken over the randomness distribution). As there are at least two different output values, the statement follows. □

In the following, we fill fix the sender's input x as one of the possible values guaranteed to exist by Corollary 4.5 and omit it from our notation. We first show the following technical claim.

Lemma 4.6. *For any $i < 4R$, let $j = i - 1 \bmod 3$. Then $\lfloor (i - j - R)/3 \rfloor = \lfloor (i + 1 - j - R)/3 \rfloor$.*

Proof. Assume that $\lfloor (i + 1 - j - R)/3 \rfloor > \lfloor (i - j - R)/3 \rfloor$. This implies $i - j - R = 2 \bmod 3$. However, $R \bmod 3 = 0$, so this implies $j = i - 2 \bmod 3$, which contradicts that $j = i - 1 \bmod 3$. □

Lemma 4.7. *For all $0 \leq r \leq R$ and $1 \leq i < 4R$,*

- *(i) party $P_{i-1 \bmod 3}$ cannot distinguish $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$,*
- *(ii) if $i < 4R - 1$ and $r \leq r_i$, party $P_{i+1 \bmod 3}$ cannot distinguish $\mathcal{E}_i^r$ and $\mathcal{E}_{i+2}^r$, and*
- *(iii) the honest party simulated by P_S cannot distinguish $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$ if $r \leq r_i$.*

Proof. We prove the claim by induction on r. The induction is anchored at $r = 0$ in which no parties send any messages. Moreover, $P_{i-1 \bmod 3}$ and (the simulated honest) P_S have the same random coins in $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$. Hence, the claim immediately follows. For the induction step, assume that the claim holds for round $r - 1 \geq 0$. By the induction hypothesis, $P_{i-1 \bmod 3}$ cannot distinguish the executions by the end of round $r - 1$. The same is true for P_S if $r \leq r_i$. As the parties have identical randomness in both executions, they can distinguish them if and only if they receive different messages from some other party in round r of $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$.

- **Proof of (i).** In $\mathcal{E}_i^r$ party $P_{i \bmod 3}$ is corrupted. It $(i + 1)$-simulates its messages to $P_{i-1 \bmod 3}$. Similarly, in $\mathcal{E}_{i+1}^r$ party $P_{i+1 \bmod 3}$ is corrupted. Hence, it i-simulates its messages to $P_{i-1 \bmod 3}$ (note that $(i + 2)$-simulates its messages to P_i). Thus, $P_{i-1 \bmod 3}$ receives the same messages from P_i and P_{i+1} in round r of $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$, respectively.
  Therefore, $P_{i-1 \bmod 3}$ can distinguish the executions if and only if it receives different messages from P_S in round r of $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$. By Lemma 4.6, either P_S sends these messages according to Π in both executions or it sends a message in neither execution. In the latter case (i.e., P_S sends no messages to P_{i-1} in round r of both $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$), the claim immediately follows. In the former case (i.e., P_S sends to P_{i-1} in round r of both $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$), it holds that $r \leq r_i + 1$ and thus $r - 1 \leq r_i$. By item (iii) of the induction hypothesis, P_S cannot distinguish executions $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$. Thus, it sends the same messages up to and including round $r - 1$ of these executions. As we have previously noted, messages in round r only depend on messages up to round $r - 1$ and the random tapes of parties. We have argued that P_{i-1} receives identical messages from P_S (and all other parties) in executions $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$ up to and including round $r - 1$. Hence, this immediately yields the desired indistinguishability of executions $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$ for party $P_{i-1 \bmod 3}$, round r, and all i.
- **Proof of (iii).** Next, we prove that P_S cannot distinguish $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$ by the end of round r if $r \leq r_i$. Again, as P_S uses the same randomness its simulation of an honest party in both executions and, by the induction hypothesis, it cannot distinguish the executions by the end of round $r - 1$. Hence, the claim follows if there is no other party sending different messages to P_S in round r of $\mathcal{E}_i^r$ and $\mathcal{E}_{i+1}^r$, respectively. The induction hypothesis rules this out for party $P_{i-1 \bmod 3}$. In $\mathcal{E}_i^r$, party $P_{i \bmod 3}$ is corrupted and $(i + 1)$-simulates its messages to P_S, which shows that P_S receives identical messages from $P_{i \bmod 3}$ in round r of both executions as well. Finally, in $\mathcal{E}_{i+1}^r$ party $P_{i+1 \bmod 3}$ is corrupted and $(i + 2)$-simulates its messages to P_S. If $i = R - 1$, this means that the sent message is identical to the one in $\mathcal{E}_i^r$. Otherwise, we can apply the induction hypothesis to see that $P_{i+1 \bmod 3}$ cannot distinguish $\mathcal{E}_{i+2}^{r-1}$ from $\mathcal{E}_i^{r-1}$. Hence, $P_{i+1 \bmod 3}$ sends the same messages in round r of $\mathcal{E}_i^r$ (honestly) and $\mathcal{E}_{i+1}^r$ (as a corrupted party). Either way, the indistinguishability claim for P_S and round r follows for all i.
- **Proof of (ii).** It remains to prove that $P_{i+1 \bmod 3}$ cannot distinguish $\mathcal{E}_{i+2}^r$ from $\mathcal{E}_i^r$ if $i < 4R - 1$ and $r \leq r_i$. Again, it suffices to consider the messages received in round r of both executions. As $r_{i+2} \geq r_i$, P_S transmits as if honest in both executions, and by the induction hypothesis

(applied both for i and $i+1$) it cannot distinguish $\mathcal{E}^{r-1}_{i+2}$ from $\mathcal{E}^{r-1}_{i}$, so its messages to $P_{i+1 \bmod 3}$ are identical. In $\mathcal{E}^{r}_{i+2}$, $P_{i-1 \bmod 3}$ is dishonest and $(i+1)$-simulates its messages to $P_{i-2 \bmod 3} = P_{i+1 \bmod 3}$, i.e., sends the same messages to $P_{i+1 \bmod 3}$ as in $\mathcal{E}^{r}_{i+1}$. Moreover, by the induction hypothesis, $P_{i-1 \bmod 3}$ cannot distinguish $\mathcal{E}^{r-1}_{i+1}$ and $\mathcal{E}^{r-1}_{i}$, implying that these messages are identical to the ones it sends in $\mathcal{E}^{r}_{i}$. Finally, in execution $\mathcal{E}^{r}_{i}$, party $P_{i \bmod 3}$ $(i-1)$-simulates its messages to party $P_{i+1 \bmod 3}$. By the induction hypothesis, P_i cannot distinguish $\mathcal{E}^{r-1}_{i+1}$ and $\mathcal{E}^{r-1}_{i+2}$. Moreover, it cannot distinguish $\mathcal{E}^{r-1}_{i-1}$ and $\mathcal{E}^{r-1}_{i+1}$, as $r-1 \leq r_i - 1 \leq r_{i-1}$. Together, this shows that the messages it sends to $P_{i+1 \bmod 3}$ in $\mathcal{E}^{r}_{i+2}$ are the same as in $\mathcal{E}^{r}_{i}$. □

Corollary 4.8. *If P_S sends a message to some party P_j in round r of execution $\mathcal{E}_{4R}$, it is the same as in round r or execution $\mathcal{E}_i$. In particular, any signature P_j receives from P_S in round r of execution $\mathcal{E}_i$, it also receives in the same round of $\mathcal{E}_{4R}$.*

Final execution. We now define a final execution $\mathcal{E}'_{4R}$ in which the sender is honest and only P_3 is dishonest and behaves exactly as in $\mathcal{E}'_{4R}$. By Lemma 4.7, P_1 and P_2 cannot distinguish $\mathcal{E}_{4R}$ and $\mathcal{E}'_{4R}$, and P_S behaves just as honest version of itself simulated by the dishonest P_S in $\mathcal{E}_{4R}$.

A subtle, but crucial point is that $\mathcal{E}'_{4R}$ is only an execution in our model if dishonest P_3 can determine the messages it sends without accessing the signing functionality, since now the adversary does not control P_S. Fortunately, by Corollary 4.8, none of the executions that P_3 needs to simulate involve any other signatures than P_S creates in $\mathcal{E}_{4R}$. As P_3 needs to determine its round-r messages at the start of round r and they depend only on signatures that were sent by P_S and randomness of other parties in rounds $r-1$ and earlier, P_3 obtains all required information in a timely fashion.

We are now in the position to wrap up the proof of the theorem. For $0 < i < 4R$, consider the probability, taken over the randomness $(\rho_1, \rho_2, \rho_3, \rho_S)$, that the honest parties $P_{i-1 \bmod 3}$ and $P_{i+1 \bmod 3}$ output different values in execution $\mathcal{E}_i$. By our assumptions on Π, this probability is at most $1/(8R)$: otherwise, the adversary can sample $\mathcal{E}_i$ by sampling randomness for the dishonest parties according to the input distribution and then realizing the respective behavior of dishonest parties. This leads to the contradiction that Π has failure probability larger than $1/(8R)$ against this randomized strategy of the adversary.

By a union bound, $P_{i-1 \bmod 3}$ and $P_{i+1 \bmod 3}$ hence output the same value in *all* executions $\mathcal{E}_i$ for $1 < i < 4R$ with probability at least $1 - (4R-2)/(8R) > 1/2 + 1/(8R)$. However, by Lemma 4.7, $P_{i-1 \bmod 3}$ cannot distinguish $\mathcal{E}_i$ from $\mathcal{E}_{i+1}$ and hence outputs the same value in both executions. Therefore, the output of honest $P_3 = P_{1-1 \bmod 3}$ in $\mathcal{E}_1$ and honest $P_{4R-2 \bmod 3}$ in $\mathcal{E}_{4R}$ is identical with probability strictly larger than $1/2 + 1/(8R)$. Because honest parties cannot distinguish $\mathcal{E}_{4R}$ from $\mathcal{E}'_{4R}$, this implies that they have the same output in $\mathcal{E}'_{4R}$ as P_2 in $\mathcal{E}_1$ with probability strictly larger than $1/2 + 1/(8R)$. Let o be the random variable which is $\perp$ if at least two honest parties output distinct values in $\mathcal{E}'_{4R}$ and which takes the value of the common output of parties in $\mathcal{E}_{4R}$ otherwise. By assumption, we have that $1/2 + 1/(8R) = Pr[o \neq \perp \wedge o = x] + Pr[o \neq \perp \wedge p \neq x]$ By our choice of x and Corollary 4.5 we also have that $Pr[o \neq \perp \wedge o = x] \leq \frac{1}{2}$. This implies that $Pr[o \neq \perp \wedge o \neq x] > \frac{1}{8}$, i.e., honest parties output a value different from x with probability strictly larger than $1/(8R)$ in $\mathcal{E}'_{4R}$. However, in $\mathcal{E}'_{4R}$ the sender P_S is honest with input x. Thus, Π violates validity with probability strictly larger than $1/(8R)$ against the adversarial strategy that samples $\mathcal{E}'_{4R}$. This contradicts our assumption that Π succeeds with probability at least $1 - 1/(8R)$, completing the proof for the special case.

The general case. Now consider the general case. Again, assume for contradiction that a protocol Π violating the claim of the theorem exists. Consider the setting of the special case and define a protocol Π' among 4 parties as follows. The sender of Π' simulates the P_S parties of Π with a PKI

setup, where the sender of Π has the same input as the one of Π'. Each of the other parties of Π' simulates between 1 and $t - s$ of the remaining parties of Π, such that the remaining parties are partitioned between them as evenly as possible. Then each party outputs the common output of their simulated parties (or the default value, if the simulated parties disagree).

Observe that corrupting the sender and one other party of Π' is equivalent to corrupting at most $s + \lceil (n-s)/3 \rceil \leq t$ parties[3] in the simulated protocol Π. Thus, all parties simulated by honest parties in Π' produce outputs satisfying consistency and validity with probability at least $1 - 1/(8R)$ within R rounds. Therefore, Π is an R-round protocol for the special case, reaching a contradiction and concluding the proof of Theorem 4.3. □

4.3 The Probability Bound is Tight up to Constants

It is worth noting that it is not an artifact of the proof strategy we employed in deriving Theorem 4.3 that it leaves room for a success probability of $1 - 1/\Omega(R)$. To the contrary, a very simple protocol similar to the one of Fitzi et al. [13] achieves this, granted access to a shared coin. For simplicity, we assume that the coin is unbiased; a biased coin reduces the success probability accordingly.

(1) The sender multicasts its signed input.
(2) For $R-1$ rounds, honest parties echo the first two signatures they receive (all other messages are ignored).
(3) Afterwards, a shared coin is used to pick a value $1 \leq r \leq R$ uniformly at random.
(4) Each honest party outputs the default value if it received no or two conflicting signed values by round r. Otherwise, the signed value received first is the output.

This protocol runs for $R + 1$ rounds. If the sender is honest, it succeeds deterministically. Otherwise, before the shared coin is evaluated, the adversary must commit to the rounds

- $1 \leq r_1 \leq R$ when the first honest party receives some signed value x (if there is no such round, set $r_1 := R + 1$; in case of a tie, use any value x that an honest party echoes) and
- $r_1 \leq r_2 \leq R$ in which the first honest party sees a signature on a different value than x (again, $r_2 := R + 1$ if there is no such round).

Observe that the echoing ensures that each honest party sees x by round $r_1 + 1$ and each honest party sees conflicting values by round $r_2 + 1$ (unless $r_1 = R$ or $r_2 = R$, respectively). Thus, the only two values of the shared coin for which the outputs of honest parties might not satisfy consistency are r_1 and r_2. As the probability that $r = r_1$ or $r = r_2$ is at most $2/R$, the protocol thus succeeds with probability at least $1 - 2/R$.

References

[1] Ittai Abraham, T.-H. Hubert Chan, Danny Dolev, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Communication complexity of byzantine agreement, revisited. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 317–326. ACM, 2019.

[2] Piyush Bansal, Prasant Gopal, Anuj Gupta, Kannan Srinathan, and Pranav Kumar Vasishta. Byzantine agreement using partial authentication. In *International Symposium on Distributed Computing*, pages 389–403. Springer, 2011.

[3] Birgit Baum-Waidner, Birgit Pfitzmann, and Michael Waidner. Unconditional byzantine agreement with good majority. In *Annual Symposium on Theoretical Aspects of Computer Science*, pages 285–295. Springer, 1991.

[4] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In Janos Simon, editor, *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*, pages 1–10. ACM, 1988.

[5] Piotr Berman, Juan A Garay, and Kenneth J Perry. Bit optimal distributed consensus. In *Computer science: research and applications*, pages 313–321. Springer, 1992.

[3] Note that $t - s \geq (n - s)/3$ is equivalent to $t - s \geq \lceil (n - s)/3 \rceil$, as t and s are both integers.

[6] Erica Blum, Jonathan Katz, Chen-Da Liu-Zhang, and Julian Loss. Asynchronous byzantine agreement with subquadratic communication. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part I*, volume 12550 of *Lecture Notes in Computer Science*, pages 353–380, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland.
[7] Malte Borcherding. On the number of authenticated rounds in byzantine agreement. In *International Workshop on Distributed Algorithms*, pages 230–241. Springer, 1995.
[8] Malte Borcherding. Partially authenticated algorithms for byzantine agreement. In *ISCA: Proceedings of the 9th International Conference on Parallel and Distributed Computing Systems*, pages 8–11, 1996.
[9] Ran Cohen, Jack Doerner, Eysa Lee, Anna Lysyanskaya, and Lawrence Roy. An unstoppable ideal functionality for signatures and a modular analysis of the dolev-strong broadcast. Cryptology ePrint Archive, Report 2024/1807, 2024.
[10] Danny Dolev and H. Raymond Strong. Authenticated algorithms for byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.
[11] Paul Feldman and Silvio Micali. Optimal algorithms for byzantine agreement. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, pages 148–161, 1988.
[12] Matthias Fitzi, Daniel Gottesman, Martin Hirt, Thomas Holenstein, and Adam Smith. Detectable byzantine agreement secure against faulty majorities. In *Proceedings of the twenty-first annual symposium on Principles of distributed computing*, pages 118–126, 2002.
[13] Matthias Fitzi, Chen-Da Liu-Zhang, and Julian Loss. A new way to achieve round-efficient byzantine agreement. In Avery Miller, Keren Censor-Hillel, and Janne H. Korhonen, editors, *PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021*, pages 355–362. ACM, 2021.
[14] Juan A. Garay and Yoram Moses. Fully polynomial byzantine agreement in t+1 rounds. In *25th Annual ACM Symposium on Theory of Computing*, pages 31–41, San Diego, CA, USA, May 16–18, 1993. ACM Press.
[15] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*, pages 51–68. ACM, 2017.
[16] S Dov Gordon, Jonathan Katz, Ranjit Kumaresan, and Arkady Yerukhimovich. Authenticated broadcast with a partially compromised public-key infrastructure. In *Symposium on Self-Stabilizing Systems*, pages 144–158. Springer, 2010.
[17] Anuj Gupta, Prasant Gopal, Piyush Bansal, and Kannan Srinathan. Authenticated byzantine generals in dual failure model. In *International Conference on Distributed Computing and Networking*, pages 79–91. Springer, 2010.
[18] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In David S. Johnson and Uriel Feige, editors, *39th Annual ACM Symposium on Theory of Computing*, pages 21–30, San Diego, CA, USA, June 11–13, 2007. ACM Press.
[19] Jonathan Katz and Chiu-Yuen Koo. On expected constant-round protocols for byzantine agreement. In Cynthia Dwork, editor, *Advances in Cryptology – CRYPTO 2006*, volume 4117 of *Lecture Notes in Computer Science*, pages 445–462, Santa Barbara, CA, USA, August 20–24, 2006. Springer Berlin Heidelberg, Germany.
[20] Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine Generals Problem. In *Concurrency: the Works of Leslie Lamport*, pages 203–226. Association for Computing Machinery, 2019.
[21] Silvio Micali, Michael O. Rabin, and Salil P. Vadhan. Verifiable random functions. In *40th Annual Symposium on Foundations of Computer Science*, pages 120–130, New York, NY, USA, October 17–19, 1999. IEEE Computer Society Press.
[22] Marshall C. Pease, Robert E. Shostak, and Leslie Lamport. Reaching Agreement in the Presence of Faults. *J. ACM*, 27(2):228–234, 1980.
[23] Birgit Pfitzmann and Michael Waidner. *Information-theoretic Pseudosignatures and Byzantine Agreement for $t \geq n/3$*. IBM, Armonk, 1996.

A Upper Bounds for BA and BC with an Unregistered Sender in the Partially Authenticated Setting

We show a simple Byzantine Agreement protocol Π_{SBA} in the partially authenticated setting that tolerates at most $t < s/2$ Byzantine parties (Fig. 3). We note that when $s \leq 2n/3$, $t < s/2$ implies $t < n/3$. Since it is well-known that an unauthenticated Byzantine Agreement protocol suffices to solve the BA problem [5], we focus on the interesting case when $s > 2n/3$.

Theorem A.1. *Assume that Π_{auth} is a deterministic protocol that solves authenticated Byzantine Agreement among s parties against up to t Byzantine corruptions whenever $2t < s$. Then the above*

Simple Byzantine Agreement Protocol Π_{SBA}

Input and Initialization. Each party $P_i \in \mathcal{P}$ has input m_i.

Consensus of Registered Parties. Each registered party P_i ($\in \mathcal{S}$): participate in a deterministic authenticated Byzantine Agreement Protocol (denoted as Π_{auth}) with input m_i, denote the output of P_i as b_i.

Message Dissemination. Each registered party P_i: sign b_i and send $(b_i, \text{sign}(b_i, i))$ to each unregistered party P_j ($\in \mathcal{N}$).

Majority Output Rule.

- Each unregistered party P_j: denote the majority value (with any prefixed tie-breaking rule) received by P_j in the Message Dissemination phase to be v_j, output v_j.
- Each registered party P_i: output b_i.

Fig. 3. Byzantine Agreement Protocol Π_{SBA}

protocol Π_{SBA} solves Byzantine Agreement in the partially authenticated setting against up to t Byzantine corruptions whenever $2t < s$.

Proof. *Termination.* Since $2t < s$, the authenticated protocol must be secure. By termination of Π_{auth}, every honest party P_i in $\mathcal{S}$ outputs v_i in finite rounds. Since the network is synchronous, the Message Dissemination step takes one round, and thus every honest party can compute the majority and output based on the Majority Output Rule.

Agreement. Since $2t < s$, by the agreement property of Π_{auth}, all honest registered parties output the same bit b. Hence at least $s - t$ messages sent by parties in $\mathcal{S}$ equal b, while at most t messages can equal $b' \neq b$. Because $s - t > t$ (equivalently, $s > 2t$), the strict majority among the s values is b, and therefore every honest party outputs b.

Validity. If all honest parties start with the same bit m, then in particular all honest registered parties start with m. Since $2t < s$, the protocol Π_{auth} is secure, by validity of Π_{auth}, all honest registered parties output $b = m$, and by the above majority argument, every honest unregistered party P_j outputs m. □

Discussion. Independently, ignoring the PKI and running any deterministic unauthenticated BA protocol among all n parties yields feasibility when $3t < n$. Combining these two approaches implies that BA is achievable for $t \leq \max\{\lceil s/2 \rceil, \lceil n/3 \rceil\} - 1$.

Corollary A.2. *There exists a Byzantine Agreement protocol in the partially authenticated setting where s parties have PKI setups that tolerates $t \leq \max\{\lceil s/2 \rceil, \lceil n/3 \rceil\} - 1$ Byzantine parties.*

We note that the Agreement protocol Π_{SBA} can be easily transformed into a broadcast protocol with the same number of key-holders that tolerates the same number of Byzantine parties: the sender first sends its input to all parties, then all parties run this Π_{SBA} protocol and output Π_{SBA}'s output.

B Broadcast with a Registered Sender: The Special Case of $n - s < 3$

Theorem 3.14. *If $n - s < 3$, there exists a Byzantine Broadcast protocol that tolerates any number of corruptions.*

If $s \geq n - 1$, the proof is immediate from the paper of Dolev and Strong. So it remains to argue about the case of $s = n - 2$.

Lemma B.1. *The protocol Π_{BCS} satisfies consistency if $s = n - 2$ and $t \leq n - 2$.*

(Binary) Broadcast Protocol Π_{BCS}

Input and Initialization. Denote the Sender $P_S \in \mathcal{S}$. Each party P_i sets value $v_i = 0$ at the beginning of the protocol. If $P_i = P_S$ and $b = 1$, set $v_i \coloneqq 1$.

Gradecast. Each party P_i: participate in the Gradecast protocol with input value $b \in \{0, 1\}$ iff $P_i = P_S$. Denote the output of party P_i to be (v_i, g_i).

Weak Consensus of Non-PKI parties (among at most 2 parties).

(1) Each unregistered party $P_j \in \mathcal{N}$: send v_j to all parties $\in \mathcal{N}$.
(2) Each unregistered party $P_j \in \mathcal{N}$: set $m_j = 1$ if it received at least one $v_i = 1$ from $P_i \in \mathcal{N}$ in the last round. Otherwise, set $m_j = 0$.

Output Rule.

- Each registered party $P_i \in \mathcal{S}$: output v_i.
- Each unregistered party $P_j \in \mathcal{N}$:
  - If $g_j = 1$, output v_j.
  - Otherwise, output m_j.

Fig. 4. (Binary) Byzantine Broadcast Protocol Π_{BCS} in the partially authenticated setting (special case)

Proof. Assume the leader P_S is honest with input b, from the validity of the gradecast protocol Π_{GC} (recall that Π_{GC} is secure against up to t Byzantine parties, where $t < n$), each honest party P_j outputs (v_j, g_j) such that $v_j = v$ and $g_j = 1$. Based on the output rule of Π_{BCS}, P_j must output $v_j = 1$ at the end of the protocol Π_{BCS}. □

Lemma B.2. *The protocol Π_{BCS} satisfies consistency if $s = n - 2$ and $t \leq n - 2$.*

Proof. Suppose there exists an honest party $P_i \in \mathcal{S}$. In this case, denote its output of Π_{GC} as (v_i, g_i). From Lemma 3.9, each honest party P_j outputs (v_j, g_j) such that $g_j = 1$ at the end of the protocol Π_{GC}. From the graded consistency, all honest parties must have $v_j = v_i$. According to the output rule, each party $P_j \in \mathcal{S}$ therefore outputs $v_j = v_i$. For each party P_j in $\mathcal{N}$, we have argued above that it has $g_j = 1$. So it outputs $v_j = v_i$ from our output rule. This means that all honest parties output v_i, completing this case.

So assume instead that all parties in $\mathcal{S}$ are Byzantine. Since $s = n - 2$, and all of those parties are corrupted, $s = t = n - 2$. So the set $\mathcal{N}$ is of size 2 and both of these parties are honest. There are two cases:

- **An honest $P_j \in \mathcal{N}$ has $(v_j, g_j = 1)$.** In this case, from the graded consistency (Lemma 3.7), each honest party P_h must output (v_h, g_h) in the gradecast protocol Π_{GC} such that $v_h = v_j$. If $v_j = 0$, each honest party P_h outputs $m_h = 0$ in the non-PKI weak consensus, all honest parties (note they all belong to $\mathcal{N}$) output $v = 0$ according to our output rule of Π_{BCS}. If $v_j = 1$, from description of the non-PKI weak consensus of the protocol Π_{BCS}, each honest party P_h outputs $m_h = 1$, all honest parties output 1 according to our output rule of Π_{BCS}.
- **All honest parties $P_j \in \mathcal{N}$ have $(v_j, g_j = 1)$.** In this case, the outputs of both honest parties are fully determined by the messages they exchange in the weak consensus phase of the protocol. As they see the same messages, they output the same value.

In conclusion, the consistency is satisfied if $s = n - 2$ and $t \leq n - 2$. □

C Multivalued Byzantine Broadcast

We construct multivalued Byzantine Broadcast Π_{MVBC} (Fig. 9). We first build a weaker form of multivalued gradecast, Π_{WGC} (Fig. 5). It is weak in the sense that if either the graded consistency or the special grade-1 property is not satisfied, at least two honest party must be able to extract

signatures of the sender on the different and non-zero values. Intuitively, those signatures can be used in our twisted binary gradedcast protocol Π_{TGC} (Fig. 7) to reach an binary gradedcast on whether the output of Π_{WGC} is reliable, with the special grade-1 property. We define an Extract algorithm to extract the signature on value v from a k-chain C_k on value v received in super round k. Specifically, $\mathsf{Extract}(C_k) = (v, \mathsf{sign}_{P_S}(v))$.

Multivalued Weak Gradecast Protocol Π_{WGC}

Input and Initialization. Denote the Sender $P_S \in \mathcal{S}$. Each party P_i set grade $g_i = 1$ and initialize $v_i = 0$ at the beginning of the protocol. If $P_i = P_S$ and $b \neq 0$, set $v_i = b$ and $\mathsf{acc}_i^0 = 1$ (treated as accepting a valid 0-chain). Each registered party $P_i \in S$ initializes a flag $\mathsf{acc}_i^k = 0$ to indicate whether P_i has accepted a k-chain in the super round k, for $k < s$. Each party P_i initializes $\mathsf{PoS}_i = \perp$ to indicate the signature of the sender P_S that P_i has extracted. Each party then proceeds with phase 1, 2 and 3 in super round $k = 1, \ldots, s$:

Phase 1: PKI Extension. Each party $P_i \in \mathcal{S}$: if P_i has accepted a valid $(k-1)$-chain C_k of value v that does not contain the signature of P_i in super round $k-1$, create k-chain $C_k = \mathsf{sign}(C_{k-1}, i)$, send Extension message $(C_k, \mathsf{Extension})$ to all parties.

Phase 2: Echo from Non-PKI Parties.

- Each party $P_i \in \mathcal{S}$: if P_i received a valid k-chain C_k in an Extension message $(C_k, \mathsf{Extension})$ of Phase 1, if $v_i = 0$, set $v_i = v$ and $\mathsf{acc}_i^k = 1$, set $\mathsf{PoS}_i = \mathsf{Extract}(C_k)$.
- Each party $P_i \in \mathcal{N}$:
  - If P_i received a valid k-chain C_k in an Extension message $(C_k, \mathsf{Extension})$ in phase 1 and $v_i = 0$: set $v_i = v$, $\mathsf{PoS}_i = \mathsf{Extract}(C_k)$ and send Echo_1 message (C_k, Echo_1) to all other parties.
  - If P_i received a valid k-chain C_k on value v in an Extension message $(C_k, \mathsf{Extension})$ in phase 1 and $v_i = v$, $g_i = 0$: set $g_i = 1$, $\mathsf{PoS}_i = \mathsf{Extract}(C_k)$ and send Echo_1 message (C_k, Echo_1) to all other parties.

Phase 3: Chain Acceptance from $\mathcal{N}$.

(1)
- Each party $P_i \in \mathcal{S}$: if $v_i = 0$ and P_i received a valid k-chain C_k in an Echo_1 message (C_k, Echo_1) in the last round: set $v_i = v$ and $\mathsf{acc}_i^k = 1$.
- Each party $P_i \in \mathcal{N}$: if $v_i = 0$ and P_i received a valid k-chain in an Echo message (C_k, Echo_1), set $v_i = v$, $g_i = 0$, $\mathsf{PoS}_i = \mathsf{Extract}(C_k)$ and send (C_k, Echo_2) to all parties.

(2)
- Each party $P_i \in \mathcal{S}$: if $v_i = 0$ and it received a valid k-chain C_k in an Echo_2 message in the last round, set $\mathsf{acc}_i^k = 1$ and $v_i = v$, $\mathsf{PoS}_i = \mathsf{Extract}(C_k)$.
- Each party $P_i \in \mathcal{N}$: if it received a valid k-chain C_k in an Echo_2 message (C_k, Echo_2), and $v_i = 0$: set $g_i = 0$, $\mathsf{PoS}_i = \mathsf{Extract}(C_k)$ and send Echo_3 message (C_k, Echo_3) to all parties $P_j \in \mathcal{S}$.

At the end of Phase 3: For each party $P_i \in \mathcal{S}$: if it received a valid k-chain C_k in an Echo_3 message (C_k, Echo_3) in the last round and $v_i = 0$, set $\mathsf{PoS}_i = \mathsf{Extract}(C_k)$, set $\mathsf{acc}_i^k = 1$ and $v_i = v$.

Output Rule: For each party P_i: output $(v_i, g_i, \mathsf{PoS}_i)$.

Fig. 5. Weak Gradecast Protocol Π_{WGC}

First we show the value PoS_i is unique for each party P_i, if $v_i \neq 0$.

Lemma C.1. *Each honest party P_i outputs unique PoS_i if $v_i \neq 0$.*

Proof. In our Π_{WGC} protocol, each party P_i only extracts a valid chain of value v if $v_i = 0$, then P_i sets $v_i = v$. Moreover, once it has set $v_i = v$, it never sets v_i to be any other value v'. In conclusion, the value PoS_i is unique if $v_i \neq 0$. □

Lemma C.2. *There is no honest registered party $P_i \in \mathcal{S}$ that sets $\mathrm{acc}_i^s = 1$ in the super round s.*

Proof. P_i only accepts an s-chain C_s if C_s contains signatures from s different parties and it does not contain signature of P_i. Since there exist at most s parties that can sign on the chain, there exists no such C_s that P_i can accept in the super round s. □

Lemma C.3. *The protocol Π_{WGC} satisfies validity.*

Proof. If the sender P_S is honest with input v, there exists at most one value v that is signed by the sender. Each party P_i sets $v_i = v$ and $g_i = 1$ in the second phase of super round 1. Each party P_i must leave v_i unchanged from the output rule and output $(v, \mathrm{sign}_S(v))$. □

Lemma C.4. *If an honest registered party $P_i \in \mathcal{S}$ that outputs (v_i, g_i) at the end of the protocol and $v_i = 0$, for any honest party P_j that outputs (v_j, g_j), we have that $v_j = 0$.*

Proof. This means P_i has never accepted any chain in any super round k. We consider the following cases:

- **Case 1.** $P_j \in \mathcal{S}$: By Lemme C.2, P_j only accepts a chain on value v_j in the super round $k < s$. P_h must receive chain extended by P_j in the super round $k+1$ and set $v_h = v_j$ by the super round $k+1$ if $v_h = 0$. If $v_h \neq 0$ by the super round $k+1$, P_h cannot output $v_h = 0$ at the end of the protocol Π_{WGC} , contradicting to the assumption that P_h outputs with $v_h = 0$ at the end of the protocol Π_{WGC} .
- **Case 2.** $P_j \in \mathcal{N}$: Assume P_j sets $v_j \neq 0$ in the super round k. If it sets $v_j \neq 0$ in phase 2, it must send Echo_1 message to P_h and P_h must set $v_h = v_j$ and cannot output $(0, g_h)$ at the end of the protocol Π_{WGC}, contradicting to the assumption that P_h outputs $(0, g_h)$ at the end of the protocol. If P_j sets $v_j \neq 0$ in phase 3, it must send Echo_3 message to P_h and P_h must set $v_h = v_j$ and cannot output $(0, g_h)$ at the end of the protocol Π_{WGC}, contradicting to the assumption that P_h outputs with $v_h = 0$ at the end of the protocol Π_{WGC}.

In conclusion, if $v_i = 0$, there exists no honest party P_j that outputs $v_j \neq 0$. □

Lemma C.5. *If an honest registered party $P_i \in \mathcal{S}$ that outputs (v_i, g_i) at the end of the protocol, if $v_i \neq 0$, there exists no honest party P_j that outputs $v_j = 0$.*

Proof. Toward a contradiction, assume that P_j outputs $v_j = 0$. We consider the following cases:

- **Case 1.** $P_j \in \mathcal{S}$: From Lemma C.4, this cannot happen.
- **Case 2.** $P_j \in \mathcal{N}$: By Lemma C.2, P_i can only accept a chain C_k on the value v_i in the super round $k < s$, P_i must extend C_k and send Extension message in the phase 1 of super round $k+1$. If P_j has $v_j = 0$ at the time receiving the Extension message from P_i, it must set $v_j = v_i$, and it cannot output $(0, g_j)$ at the end of Π_{WGC}. Otherwise, P_j has $v_j \neq 0$ at the time P_j is receiving the Extension message from P_i, P_j cannot output $(0, g_j)$ at the end of protocol Π_{WGC}.

In conclusion either case leads to a contradiction with P_j outputting $v_j = 0$. Hence, if $v_i \neq 0$, there exists no honest party P_j that outputs $v_j = 0$. □

Lemma C.6 (Conditional graded-consistency of Π_{WGC}). *If there exist two honest parties P_i and P_j that output $(v_i, g_i, \mathrm{PoS}_i)$ and $(v_j, g_j, \mathrm{PoS}_j)$, respectively, such that $g_i = 1$ and $v_i \neq v_j$, we have there exists two honest parties P and P', that output (v, g, PoS) and (v', g', PoS'), respectively, such that $v \neq 0$, $v' \neq 0$, $v \neq v'$, $\mathrm{PoS} \neq \perp$, $\mathrm{PoS}' \neq \perp$ and the value of PoS is different from the value of PoS'.*

Proof. Assume that parties P_i and P_j are as in the Lemma statement. We consider the following cases:

- **Case 1. There exists an honest registered party $P_h \in \mathcal{S}$ (P_h can be P_i) that outputs $v_h = 0$.**
  This leads to a contradiction with Lemma C.4.
- **Case 2. There exists an honest registered party $P_h \in \mathcal{S}$ that outputs $v_h \neq 0$.**
  This leads to a contradiction with Lemma C.5.
- **Case 3. All honest parties are in the set $\mathcal{N}$.**
  - **Case 3a. $v_i = 0$, $v_j \neq 0$.** Since $g_i = 1$, P_i has not received any k-chain in any super round $k < s$. Now consider the last super round s. If any honest party $P_j \in \mathcal{N}$ that sets $v_j \neq 0$ in phase 2 or the first round of phase 3, P_j either sends an Echo_1 message or an Echo_2 message to P_i. Thus, P_i would set $v_i \neq 0$ in phase 3, contradicting to the assumption that $v_i = 0$.
  - **Case 3b. $v_i \neq 0$, $v_j = 0$.** In this case, $g_i = 1$ only happens if P_i sets $v_i \neq 0$ in phase 2 in a super round k. P_i will send an Echo_1 message to all other parties in phase 2 of super round k. Thus each party $P_j \in \mathcal{N}$ with $v_j = 0$ sets $v_j \neq 0$ in phase 3 of super round k and it cannot output $(0, g_j)$.
  - **Case 3c. $v_i \neq 0$, $v_j \neq 0$.** In this case, when P_i sets $v_i \neq 0$, it must set the value of PoS_i to be v_i, similarly, P_j must set the value of PoS_j to be v_j. From the assumption that $v_i \neq v_j$, we have value of PoS is different from the value of PoS′.

□

Lemma C.7 (Conditional grade-1 property of Π_{WGC}). *If there exists an honest party $P_i \in \mathcal{S}$ (denote its output as $(v_i, g_i, \mathsf{PoS}_i)$), and an honest party P_j that outputs $(v_j, g_j, \mathsf{PoS}_j)$ such that $g_j = 0$, we have that there exists two honest parties P and P', that output (v, g, PoS) and (v', g', PoS'), respectively, such that $v \neq 0$, $v' \neq 0$, $v \neq v'$, $\mathsf{PoS} \neq \perp$, $\mathsf{PoS}' \neq \perp$ and the value of PoS is different from the value of PoS'.*

Proof. We consider/rule out all of the following cases:

- **Case 1. $v_i \neq 0$.** By Lemma C.5, there exists no honest party P_j that outputs $(0, g_j)$. By Lemma C.2, P_i can only accept a k-chain leading to setting $v_i \neq 0$ in super round $k < s$. Thus, P_i sends an Extension message in super round $k + 1$. We distinguish two cases, both of which lead to a contradiction:
  - **Case 1a. $v_j = v_i$.** In this case, P_j must set $g_j = 1$ in the phase 2. Thus, it cannot output grade $g_j = 0$ at the end of the protocol.
  - **Case 1b. $v_j \neq v_i$.** In this case, from the assumption we have $v_j \neq 0$ and $v_i \neq 0$, from the uniqueness of algorithm PoS (Lemma C.1), we must have $\mathsf{PoS}_i \neq \mathsf{PoS}_j$, $\mathsf{PoS}_i \neq \perp$ and $\mathsf{PoS}_j \neq \perp$.
- **Case 2. $v_i = 0$.** This leads to the two following cases:
  - **Case 2a. $v_j = 0$.** In this case, P_j can set g_j to be 0 only if P_j received a valid k-chain in the second round of phase 3. It must send Echo_3 message to all parties in $\mathcal{S}$. In particular, P_i must receive the Echo_3 message and set $v_i \neq 0$, contradicting the assumption that $v_i = 0$.
  - **Case 2b. $v_j \neq 0$.** In this case, P_j can set g_j to be 0 only if P_j received a valid k-chain in the first round of phase 3. It must send Echo_2 message to all parties in $\mathcal{S}$. In particular, P_i must received the Echo_2 message and set $v_i \neq 0$, contradicting the assumption that $v_i = 0$.

□

Similar to the binary case, we build Weak (multivalued) Broadcast Protocol Π_{WBC} from Π_{WGC}, assuming a deterministic unauthenticated BA protocol Π_{unau} among $|\mathcal{N}|$ unregistered parties.

Π_{WBC} is weak in the sense that the consistency is conditional: either the consistency is satisfied, or there exists two honest parties that hold sender's signature on different values.

Weak Broadcast Protocol Π_{WBC}

Input and Initialization. Denote the Sender $P_S \in \mathcal{S}$. Each party sets value $v_i = 0$ at the beginning of the protocol. If $P_i = P_S$, P_i sets $v_i = b$.

Weak Gradecast. Each party P_i participates in the Weak Gradecast protocol Π_{WGC} with Sender $P_S \in \mathcal{S}$ and input value b. Denote the output of party P_i in the protocol Π_{WGC} to be (v_i, g_i, PoS_i).

Consensus of Non-PKI parties.

- Each unregistered party $P_i \in \mathcal{N}$: input v_i to the unauthenticated BA Π_{unau}, denote the output of P_i as m_i.
- Each registered party $P_i \in \mathcal{S}$: do not participate in the deterministic unauthenticated BA and wait for it to terminate; we use a fixed-round unauthenticated BA (e.g., the one in [5], which takes $3(\lceil (n-s)/3 \rceil - 1) + 1$ rounds).

Output Rule.

- Each registered party $P_i \in \mathcal{S}$: output v_i.
- Each unregistered party $P_j \in \mathcal{N}$:
  - If $g_j = 1$, output (v_j, PoS_j).
  - Otherwise
    - if $m_j = 1$, output (v_j, PoS_j).
    - if $m_j = 0$, output $(0, \text{PoS}_j)$.

Fig. 6. Weak Broadcast Protocol Π_{WBC} in the partially authenticated setting

Lemma C.8 (Validity of Π_{WBC}). *If the sender is honest and it starts with input b, each honest party P_j must output (v_j, PoS_j) such that $v_j = b$.*

Proof. If the sender is honest and it starts with input b, from the validity of Π_{WGC}, each honest party P_j must output (v_j, g_j, PoS_j) such that $v_j = b$ and $g_i = 1$, so they must output $v_j = b$ according to the output rule. □

Lemma C.9 (Conditional Consistency of Π_{WBC}). *If there are at most t Byzantine parties, where $t \leq s + \lceil (n-s)/3 \rceil - 1$, and two honest parties P_i and P_j that output (v_i, PoS_i) and (v_j, PoS_j), respectively, such that $v_i \neq v_j$, then there exist party P and party P', that output (v, PoS) and (v', PoS'), respectively, such that $\text{PoS} \neq \perp$, $\text{PoS}' \neq \perp$, and $\text{PoS} \neq \text{PoS}'$.*

Proof. We consider the following cases:

- **Case 1. There exists an honest registered party $P_h \in \mathcal{S}$.** Denote the output of P_h in Π_{WGC} as (v_h, g_h, PoS_h). From the conditional grade-1 property and conditional graded consistency (Lemma C.7, Lemma C.6), we must have either each honest party P_w outputs (v_w, g_w, PoS_w) such that $v_w = v_h$, or there exists two parties P and P' that outputs (v, g, PoS) and (v', g', PoS'), respectively, such that $\text{PoS} \neq \perp$, $\text{PoS}' \neq \perp$ and the value of PoS is different from the value of PoS′.
- **Case 2. All registered parties are Byzantine.** Since $t \leq s + \lceil (n-s)/3 \rceil - 1$, we have $3(t-s) \leq 3\lceil (n-s)/3 \rceil - 3 < n - s$. If all parties in $\mathcal{S}$ are Byzantine, then there are at most $t - s$ Byzantine parties in $\mathcal{N}$, and therefore the unauthenticated BA among parties in $\mathcal{N}$ must be secure. As a result, all honest parties in the set $\mathcal{N}$ must have the same output m for the unauthenticated BA. We consider the following cases:

- **Case 2a. There exists an honest party P_h that outputs $(v_h, g_h, \mathrm{PoS}_h)$ in the Π_{WGC} protocol such that $g_h = 1$.** From conditional graded consistency (Lemma C.6), either we have each honest party P outputs (v, g, PoS), such that $v = v_h$, or there exists two honest parties that output different and non-$\perp$ POS (Proof of Signature) values. Since the unauthenticated BA is secure and since it satisfies validity, all honest parties must output $m = v_h$ in the unauthenticated BA protocol. Each honest party P' must output (v', PoS') such that $v' = v$.
- **Case 2b. Each honest party P_h outputs $(v_h, g_h, \mathrm{PoS}_h)$ in the Π_{WGC} protocol such that $g_h = 0$.** In this case, it holds from Lemma C.6 that either each honest party P_w $(v_w, g_w, \mathrm{PoS}_w)$ outputs $v_w = v_h$, and by validity of the unauthenticated BA, they must output the same value, or there exists two honest parties P and P' that output (v, PoS) and (v', PoS'), respectively, such that $\mathrm{PoS} \neq \perp$, $\mathrm{PoS}' \neq \perp$, and $\mathrm{PoS} \neq \mathrm{PoS}'$.

□

Lemma C.10 (Termination of Π_{WBC}). *The weak broadcast protocol Π_{WBC} terminates in finite rounds.*

Proof. The weak gradedcast protocol Π_{WGC} terminates in $4s$ rounds. The unauthenticated BA terminates in $3((\lceil (n-s)/3 \rceil) - 1) + 1$ rounds. In total, the protocol Π_{WBC} terminates in finite rounds. □

Now we introduce an important definition: A Proof of Equivocation (PoE) chain.

Definition C.11 (k-PoE Chain with respect to Sender P_S). A *k-PoE chain* E_k (for $k > 0$) with respect to sender P_S is recursively defined as follows:

- a 1-PoE chain for sender P_S is a pair $((v, \mathsf{sign}_{P_S}(v)), (v', \mathsf{sign}_{P_S}(v')))$ such that $v \neq v'$, and
- for $k > 1$, a k-PoE chain on v is a pair $(E_{k-1}, \mathsf{sign}(E_{k-1}, i))$, where E_{k-1} is a $(k-1)$-POE chain.

A k-PoE chain E_k is *valid* for P_i if

- E_k contains a valid E_1 chain.
- In E_k, all $k-1$ signers are distinct.
- E_k does not contain a signature of P_i (this trivially holds for an unregistered party $P_i \in \mathcal{N}$).

Now we build a twisted binary gradecast protocol Π_{TGC} (Fig. 7). It is a twisted version of Π_{GC} in which, in the initialization, each party $P_i \in \mathcal{P}$ inputs a signature of the sender PoS_i (Proof of Signature). The main difference compared with our Π_{GC} protocol is that we begin with a designated phase in which parties P_i exchange their values of PoS_i. Their aim is to gather two signatures of the sender on distinct values, so as to serve as a PoE (Proof of Equivocation). From here on out, Π_{TGC} closely resembles our binary gradecast protocol. Starting from phase 2 of the super round 1, only PoE chains are extended and delivered. The (twisted) validity of Π_{TGC} requires that if there exist signatures of the sender on different values held by any two honest parties, each honest party outputs 1 with grade 1. Intuitively, this scenario is equivalent to the one where the sender inputs 1 in the Π_{GC} protocol. Moreover, we require that if the sender is honest, which means there cannot exist any PoE chain, each party outputs 0 with grade 1. Informally, this resembles the case where the sender P_S in Π_{GC} starts with input 0.

The following Lemmas follows a similar pattern as in the proof of Π_{GC}.

Lemma C.12. *There is no party $P_i \in \mathcal{S}$ that sets $\mathrm{acc}_i^s = 1$ in the super round s.*

(Binary) Gradecast Protocol Π_{TGC}

Input and Initialization. Denote the Sender $P_S \in \mathcal{S}$, each party P_i sets grade $g_i = 1$ and value $v_i = 0$, inputs PoS_i at the beginning of the protocol. Each party $P_i \in \mathcal{S}$ initializes a flag $\mathsf{acc}_i^k = 0$ to indicate whether P_i has accepted a k-PoE chain E_k in the super round k, for $k < s$. Each party then proceeds with Phases 1, 2 and 3 in super round $k = 1, \ldots, s$:

Phase 1: PKI Extension.

- For $k = 1$, each party P_i: send PoS_i to all other parties.
- For $k = 2, \ldots, s$: each party $P_i \in \mathcal{S}$: if $\mathsf{acc}_i^{k-1} = 1$ (which means P_i has accepted a valid $(k-1)$- PoE chain in super round $k-1$), extend E_{k-1} by signing it to obtain k-PoE chain $E_k = \mathsf{sign}(E_{k-1}, i)$ and send Extension message $(E_k, \mathsf{Extension})$ to all parties.

Phase 2: Echo from Non-PKI Parties.

- If $k = 1$: each party P_i: if P_i received signatures of the signer P_S on different values $(v, \mathsf{sign}_{P_S}(v))$ and $(v', \mathsf{sign}_{P_S}(v'))$ in the last round, for some v and v' such that $v \neq v'$, set $E_1 = ((v, \mathsf{sign}_{P_S}(v)), (v', \mathsf{sign}_{P_S}(v')))$, set $v_i = 1$. Meanwhile,
  - Each registered party $P_i \in \mathcal{S}$: set $\mathsf{acc}_j^1 = 1$ to indicate P_i has accepted the valid 1-PoE chain in the super round 1.
  - Each unregistered party $P_i \in \mathcal{N}$: send Echo_1 message (E_1, Echo_1) to all parties.
- If $k = 2, \ldots, s$:
  - Each registered party $P_i \in \mathcal{S}$ that holds a valid k-PoE chain E_k in an Extension message $(E_k, \mathsf{Extension})$: if $v_i = 0$, set $v_i = 1$ and $\mathsf{acc}_i^k = 1$.
  - Each unregistered party $P_i \in \mathcal{N}$ that obtained a valid k-PoE chain E_k from an Extension message $(E_k, \mathsf{Extension})$:
    - if $v_i = 0$, set $v_i = 1$ and send Echo_1 message (E_k, Echo_1) to all parties.
    - if $v_i = 1$ and $g_i = 0$, set $g_i = 1$ and send Echo_1 message (E_k, Echo_1) to all parties.

Phase 3: Chain Acceptance from $\mathcal{N}$.

(1)
- Each registered party $P_i \in \mathcal{S}$: if $v_i = 0$ and P_i received a valid k-PoE chain E_k in an Echo_1 message (E_k, Echo_1) in Phase 2, set $v_i = 1$ and $\mathsf{acc}_i^k = 1$.
- Each unregistered party $P_i \in \mathcal{N}$: if $v_i = 0$ and it received a valid k-PoE chain in an Echo_1 message (E_k, Echo_1) in Phase 2, set $v_i = 1$ and $g_i = 0$ and send (E_k, Echo_2) to all parties.

(2)
- Each registered party $P_i \in \mathcal{S}$: if $v_i = 0$ and it received a valid k-PoE chain E_k in an Echo_2 message in the last round, set $\mathsf{acc}_i^k = 1$ and $v_i = 1$.
- Each unregistered party $P_i \in \mathcal{N}$: if it received a valid k-PoE chain E_k in an Echo_2 message (E_k, Echo_2) received in the last round and $v_i = 0$, set $g_i = 0$ and send Echo_3 message (E_k, Echo_3) to all parties $P_j \in \mathcal{S}$.

At the end of Phase 3: Each registered party $P_i \in \mathcal{S}$: if it received a valid k-PoE chain E_k in an Echo_3 message (E_k, Echo_3) in the last round and $v_i = 0$, set $\mathsf{acc}_i^k = 1$ and $v_i = 1$.

Output Rule. At the end of super round s. Each party P_i: output (v_i, g_i).

Fig. 7. (Binary) Twisted Gradecast Protocol Π_{TGC}

Proof. P_i only accepts an s-chain E_s if E_s contains signatures from s different parties and it does not contain signature of P_i. Since there exist at most s parties that can sign on the chain, there exists no such E_s that P_i can accept in the super round s. □

Lemma C.13. *If there is an honest party $P_i \in \mathcal{S}$ that outputs (v_i, g_i) such that $v_i = 1$, then $v_j = 1$ for any honest party P_j that outputs (v_j, g_j).*

Proof. Since each party $P_i \in \mathcal{S}$ outputs v_i only if it has accepted a k-PoE chain in super round k, from Lemma C.12, the accepted chain must be a k-PoE chain for $k < s$. Therefore, P_i creates

a valid $(k+1)$-PoE chain E_{k+1} and sends $(E_{k+1}, \text{Extension})$ to all parties in the phase 1 in super round $k+1$. Each party P_j with $v_j = 0$ sets $v_j = 1$ in the first round of phase 2 of super round $k+1$. Furthermore, P_j never sets $v_j = 0$ once it has $v_j = 1$ according to the protocol, so P_j must output $v_j = 1$ at the end of the protocol Π_{TGC}. □

Lemma C.14. *If there is an honest party $P_i \in \mathcal{S}$ that outputs (v_i, g_i) such that $v_i = 0$, then $v_j = 0$ for any honest party P_j that outputs (v_j, g_j).*

Proof. Assume $P_i \in \mathcal{S}$ outputs (v_i, g_i) such that $v_i = 0$. This means it has not accepted any PoE chain during any super round of the protocol. Toward a contradiction, let P_j be an honest party that sets $v_j = 1$. We consider two cases: first, if $P_j \in \mathcal{S}$, then by Lemma C.13, P_i must output $v_i = 1$ at the end of the protocol, contradicting to the assumption that $v_i = 0$. In the second case, we have $P_j \in \mathcal{N}$. Then, P_j sets $v_j = 1$ in super round k for some $k < s$. Since the k-PoE chain E_k P_j received cannot contain the signature of P_i (as we have assumed that it never accepts a chain in any super round), $k < s$. There are the following sub-cases:

- If $P_j \in \mathcal{N}$ sets $v_j = 1$ in phase 2 of super round k, P_i must receive the Echo_1 message containing E_k of P_j in phase 2 of super round k. P_i sets $v_i = 1$ and outputs (v_i, g_i) with $v_i = 1$ at the end of the protocol.
- If $P_j \in \mathcal{N}$ sets $v_j = 1$ in phase 3 of super round k, P_i must receive the PoE chain either from an Echo_2 message or Echo_3 message from P_j in phase 3 of super round k. Upon receiving the k-PoE chain E_k as part of one of those messages, P_i sets $v_i = 1$ and outputs (v_i, g_i) with $v_i = 1$ at the end of the protocol.

Clearly, both of these cases lead to a contradiction with P_i outputting $(v_i = 0, g_i)$. Therefore, in conclusion, each party P_j must output (v_i, g_i) such that $v_i = 0$ at the end of the protocol. □

Lemma C.15. *If there is an honest registered party $P_i \in \mathcal{S}$ that outputs (v_i, g_i), each honest party P_j outputs (v_j, g_j) with $v_j = v_i$.*

Proof. The proof follows directly from Lemma C.13 and Lemma C.14. □

Lemma C.16 (Validity of Π_{TGC}). *If any two honest parties P_i and P_j that input PoS_i and PoS_j on different values at the beginning of the protocol Π_{TGC}, each honest party P_j outputs (v_j, g_j) such that $v_j = 1$ and $g_j = 1$ at the end of the protocol. Moreover, if the sender P_S is honest, each honest party P_i outputs (v_i, g_i) such that $v_i = 0, g_i = 1$ at the end of the protocol.*

Proof. If any two honest parties P_i and P_j that input PoS_i and PoS_j on different values at the beginning of the protocol Π_{TGC}, each honest party P_h must set $v_h = 1$ in the phase 2 of the super round 1. They must output (v_h, g_h) such that $v_h = 1$ and $g_h = 1$ at the end of Π_{TGC}. If the sender P_S is honest, since we assume an ideal signature scheme, there exists no proof of equivocation, all honest parties must output $(0, 1)$ at the end of the protocol Π_{TGC}. □

Lemma C.17 (Graded Consistency of Π_{TGC}). *The protocol Π_{TGC} satisfies graded consistency, which means if any honest party P_i that outputs (v_i, g_i) at the end of the protocol Π_{TGC}, such that $g_i = 1$, for any honest party P_j that outputs (v_j, g_j) at the end of the protocol, we must have $v_i = v_j$.*

Proof. Assume an honest party P_i outputs (v_i, g_i) such that $g_i = 1$. If there exists an honest registered party P_j (P_j could be P_i) $\in \mathcal{S}$, from Lemma C.15, all honest parties must output the same value as v_j. So assume instead that all honest parties are in the set $\mathcal{N}$. There are following different cases:

- Case $v_i = 1$. In this case, $g_i = 1$ only happens if P_i sets $v_i = 1$ in phase 2 in a super round k. P_i will send an Echo_1 message to all other parties in phase 2 of super round k. Thus, each party $P_j \in \mathcal{N}$ sets $v_j = 1$ in phase 3 of super round k and output (v_j, g_j) such that $v_j = 1$.

- Case $v_i = 0$. In this case, $g_i = 1$ only happens if there is no honest party P_j that sets $v_j = 1$ in any super round $k < s$. Now consider the last super round s. If any honest party $P_j \in \mathcal{N}$ sets $v_j = 1$ in phase 2 or the first round of phase 3, P_j either sends an Echo_1 message or an Echo_2 message to P_i. Thus, P_i would set $v_i = 1$ in phase 3, which is a contradiction. As a result, each honest party P_j must output (v_j, g_j) such that $v_j = 0$ at the end of the protocol.

In conclusion, the protocol Π_{TGC} satisfies Graded Consistency. □

Lemma C.18 (Grade-1 property of Π_{TGC}). *The protocol Π_{TGC} satisfies a special grade-1 property, which means if there exists an honest party P_i in the set $\mathcal{S}$, for any honest party P_j that outputs (v_j, g_j), we must have $g_j = 1$.*

Proof. Assume an honest party $P_i \in \mathcal{S}$ outputs (v_i, g_i) at the end of the protocol. Throughout the protocol, $P_i \in \mathcal{S}$ only outputs with grade $g_i = 1$. Now we distinguish two cases for the value v_i and an arbitrary honest party $P_j \in \mathcal{N}$:

- If $v_i = 1$, from Lemma C.12, P_i only accepts a k-PoE chain and sets $v_i = 1$ in super round $k < s$. Since P_i only accepts a PoE chain which it isn't itself a part of, it can extend the PoE chain by adding its own signature to it and send an Extension message $(E_{k+1}, \text{Extension})$ for the extended $(k+1)$-chain E_{k+1} in super round $k+1$. Thus, each honest party P_j sets $v_j = 1$ and it must have $g_j = 1$ in phase 2 of super round $k+1$.
- If $v_i = 0$, assume there is an honest party $P_j \in \mathcal{N}$ that outputs (v_j, g_j) with $g_j = 0$. Since $g_i = 1$, by graded consistency (Lemma C.17), P_j must output $v_j = 0$. P_j only sets $g_j = 0$ in phase 3 of some super round $k \le s$, upon receiving a valid k-PoE chain E_k in an Echo_2 message. If $k = s$, the PoE chain must contain the signature of P_i. Thus $v_i = 1$, contradicting the assumption that $v_i = 0$. If $k < s$, P_j sends an Echo_3 message containing the k PoE-chain E_k to every party in $\mathcal{S}$, including P_i. Therefore, P_i sets $v_i = 1$ at the end of Phase 3 of super round $k < s$, contradicting the assumption that $v_i = 0$.

□

Lemma C.19 (Termination of Π_{TGC}). *The protocol Π_{TGC} terminates in finite rounds.*

Proof. The protocol Π_{TGC} terminates in $4s$ rounds. □

Combining Π_{WBC} and Π_{TGC}, we finally build a multivalued gradecast protocol Π_{MVGC} (Fig. 8).

Multivalued Gradecast Protocol Π_{MVGC}

Input and Initialization. Denote the Sender $P_S \in \mathcal{S}$. Each party P_i sets grade $g_i = 1$ and value $v_i = 0$ at the beginning of the protocol. P_i inputs value b if $P_i = P_S$. **Weak Broadcast Π_{WBC}.** Each party P_i participates in the Weak Broadcast protocol Π_{WBC} with Sender $P_S \in \mathcal{S}$, input value b if $P_i = P_S$. Denote the output of party P_i to be (v_i, PoS_i).

Twisted Gradecast Π_{TGC}. Each party P_i: participate in the Π_{TGC} protocol with inputs PoS_i, denote the output of party P_i as (v_i', g_i').

Output Rule. Each party P_i: if P_i outputs $(1, g_i')$ in the Π_{TGC} protocol, then P_i outputs $(0, g_i')$. If P_i outputs $(0, g_i')$ in the Π_{TGC} protocol, P_i outputs (v_i, g_i').

Fig. 8. Multivalued Gradecast Protocol Π_{MVGC} in the partially authenticated setting

Lemma C.20 (Validity of Π_{MVGC}). *If the sender P_S is honest has input b, each honest party P_i must output $(v_i = b, g_i = 1)$ from protocol Π_{MVGC}.*

Proof. Assume the sender is honest. From the validity of Π_{WBC} (Lemma C.8), each honest party P_i must output $(v_i = v, \mathsf{PoS}_i)$. We note that no PoE can form in Π_{TGC}. From the validity of Π_{TGC} (Lemma C.16), party P_i outputs $(v'_i = 0, g'_i = 1)$. From the output rule, P_i outputs $(v, 1)$ at the end of the protocol Π_{MVGC}. □

Lemma C.21 (Graded Consistency of Π_{MVGC}). *If there exists an honest party P_i that outputs (v_i, g_i) such that $g_i = 1$, each honest party P_j must output (v_j, g_j) such that $v_j = v_i$.*

Proof. Since $g_i = 1$, P_i must output $g'_i = 1$ in the Π_{TGC} protocol. We consider the following cases:

- If $v'_i = 1$, from the graded consistency of Π_{TGC} (Lemma C.17), we have $v'_j = 1$ for all honest P_j. By the output rule, P_i outputs $(0, 1)$, whereas P_j outputs $(0, g'_j)$. Thus, the graded consistency is satisfied.
- If $v'_i = 0$, from the graded consistency of Π_{TGC} (Lemma C.17), we have $v'_j = 0$. So P_i and P_j output their respective outputs v_i and v_j of Π_{WBC}. From the conditional consistency of Π_{WBC} (Lemma C.9), we must have either
  - $v_i = v_j$, where they output the same value in Π_{MVGC}.
  - there exists two honest parties P and P' that output (v, PoS) and (v, PoS'), respectively, such that $\mathsf{PoS} \neq \perp$, $\mathsf{PoS}' \neq \perp$ and the value of PoS is different from the value of PoS'. From the validity of Π_{TGC}, all honest parties must output 1, in particular, party P_i must output with $v'_i = 1$, contradicting to the assumption that $v'_i = 0$.

□

Lemma C.22 (Termination of Π_{MVGC}). *The protocol Π_{MVGC} terminates in finite rounds.*

Proof. It follows directly from Lemma C.10 and Lemma C.19, that the protocol Π_{MVGC} terminates in $4s + 4s + 3((\lceil (n-s)/3 \rceil) - 1) + 1$ rounds and it is finite. □

Lemma C.23 (Grade-1 Property of Π_{MVGC}). *If there exists an honest party $P_i \in \mathcal{S}$, for any honest party P_j that outputs (v_j, g_j) in Π_{MVGC}, we must have $g_j = 1$.*

Proof. Let (v'_i, g'_i) be the output of P_i in Π_{TGC}. From the grade-1 property of Π_{TGC} (Lemma C.18), we must have $g'_j = 1$. From the output rule of Π_{MVGC}, we must have $g_j = g'_j$, so $g_j = 1$. □

We build multivalued Broadcast Protocol Π_{MVBC} (Fig. 9) based on multivalued gradecast protocol Π_{MVGC}.

Assuming there are up to $t \leq s + \lceil (n-s)/3 \rceil - 1$ Byzantine parties, we have following Lemmas:

Lemma C.24 (Validity of Π_{MVBC}). *If the sender P_S is honest and it starts with input b, each honest party P_h outputs b at the end of the protocol Π_{MVBC}.*

Proof. If the sender P_S is honest and starts with input b, from the validity of Π_{MVGC} (Lemma C.20), any honest party P_h must output $(b, 1)$ in Π_{MVGC}. From the validity of Π_{WBC} (Lemma C.8), it must output $(b, 1)$ in Π_{WBC}. From the output rule of Π_{MVBC}, P_h must output b at the end of Π_{MVBC}. □

Lemma C.25 (Consistency of Π_{MVBC}). *Assume an honest party P_i outputs b_i at the end of the protocol Π_{MVBC}, each honest party P_j must output b_j such that $b_j = b_i$ at the end of the protocol Π_{MVBC}.*

Proof. We consider the following cases:

Multivalued Broadcast Protocol Π_{MVBC}

Input and Initialization. Denote the Sender $P_S \in \mathcal{S}$, each party P_i initializes value $v_i = 0$, set $v_i = b$ if $P_i = P_S$.

Multivalued Gradecast Protocol Π_{MVGC}. Each party P_i: participate in the Multivalued Gradecast Protocol Π_{MVGC} with Sender $P_S \in \mathcal{S}$, input value b. Denote the output of P_i as (v_i, g_i).

Unauthenticated BA Π_{unau}.

- Each unregistered party $P_i \in \mathcal{N}$: participate in the unauthenticated BA Π_{unau} (e.g., the one in [5]) with input v_i, denote the output as m_i.
- Each registered party $P_i \in \mathcal{S}$: do not participate in the deterministic unauthenticated BA and wait for it to terminate; we use a fixed-round unauthenticated BA (e.g., the one in [5], which takes $3(\lceil (n-s)/3 \rceil - 1) + 1$ rounds).

Output Rule.

- Each party $P_i \in \mathcal{S}$: output v_i.
- Each party $P_j \in \mathcal{N}$:
  - If $g_j = 1$, output v_j.
  - Otherwise ($g_j = 0$), output m_j.

Fig. 9. Multivalued Broadcast Protocol Π_{MVBC} in the partially authenticated setting

- **There exists an honest party $P_h \in \mathcal{S}$.** In this case, from the grade-1 property of Π_{MVGC} (Lemma C.23), P_i must output (v_i, g_i) in the protocol Π_{MVGC} such that $g_i = 1$. From the graded consistency of Π_{MVGC} (Lemma C.21), we must have $v_j = v_i$, for any honest party P_j. From the output rule, P_j must output $b_j = v_i$ at the end of protocol Π_{MVBC}, and b_i must equal to v_i. In conclusion, we have $b_i = b_j$.
- **There exists no honest party in the set $\mathcal{S}$.** Since $3(t-s) \leq 3\lceil (n-s)/3 \rceil - 3 < n - s$, the unauthenticated BA must be secure. We consider the following cases:
  - **If there exists one honest party P_i that outputs (v_i, g_i) in the Π_{MVGC} such that $g_i = 1$.** From the graded consistency of Π_{MVGC} (Lemma C.21), we have that each honest party P_j must output (v_j, g_j) such that $v_j = v_i$. From the consistency of unauthenticated BA, we must have $m_i = m_j$. From the output rule, P_i and P_j must output the same value.
  - **If each honest party P_i outputs (v_i, g_i) in the Π_{MVGC} with $g_i = 0$.** From the validity of unauthenticated BA, we have $m_i = 1$ and that each honest P_j must output $m_j = 1$ in the unauthenticated BA. From the output rule, P_i must output $b_i = 0$ and P_j must output $b_j = 0$.

□

Lemma C.26 (Termination of Π_{MVBC}). Π_{MVBC} *terminates in finite rounds.*

Proof. By Lemma C.22, Π_{MVGC} terminates in $4s + 4s + 3((\lceil (n-s)/3 \rceil) - 1) + 1$ rounds. Moreover, the unauthenticated BA terminates in $3((\lceil (n-s)/3 \rceil) - 1) + 1$. In total, Π_{MVBC} terminates in $8s + 6((\lceil (n-s)/3 \rceil) - 1) + 2$ rounds. □

Theorem C.27. *The Protocol Π_{MVBC} is a Multivalued Broadcast Protocol that tolerates $t \leq s + \lceil (n-s)/3 \rceil - 1$ Byzantine parties, given s registered parties (including the sender).*

Proof. The proof directly follows from Lemma C.24, Lemma C.25 and Lemma C.26. □